Học Viện Công Nghệ Bưu Chính Viễn Thông
Khoa Công Nghệ Thông Tin 2

--------oOo--------



BÁO CÁO TÌM HIỂU CHUYÊN ĐỀ

JAVA DATABASE PROGRAMMING

Giảng viên hướng dẫn:	Nguyễn Thị Tuyết Hải
Học phần:	Lập trình hướng đối tượng
Lớp:	D24CQCN01-N

Họ và tên sinh viên	**Mã số sinh viên**
- Lê Nguyễn Nhật Quang	N24DCCN066
- Bạch Quốc Thoại	N24DCCN079
- Nguyễn Thị Tường Vy	N24DCCN092
- Tăng Thành Tài	N24DCCN070
- Trần Thị Hồng Vân	N24DCCN089

Thành Phố Hồ Chí Minh , ngày 17 tháng 10 năm 2026

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU **1**
1.1 Bối cảnh và lý do chọn đề tài 1
1.2 Mục tiêu tìm hiểu 1
1.3 Phạm vi và giới hạn 2
1.4 Phương pháp thực hiện 2
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT **3**
2.1 Cơ sở dữ liệu quan hệ và ngôn ngữ SQL 3
2.1.1 Mô hình dữ liệu quan hệ 3
2.1.2 Ngôn ngữ truy vấn SQL 4
2.2 Kiến trúc JDBC 4
2.3 Các interface cốt lõi và quy trình truy cập CSDL 5
2.3.1 Nạp driver và mở kết nối 6
2.3.2 Thực thi câu lệnh và duyệt ResultSet 6
2.3.3 Giao dịch và chế độ tự động xác nhận 6
2.4 PreparedStatement và CallableStatement 7
2.4.1 PreparedStatement 7
2.4.2 CallableStatement 7
2.5 Siêu dữ liệu 8
2.6 JDBC dưới góc nhìn lập trình hướng đối tượng 8
2.6.1 Trừu tượng hóa và đa hình qua interface 8
2.6.2 Kế thừa giữa các interface 9
2.6.3 Reflection và cơ chế tự động phát hiện driver 9
2.6.4 Xử lý ngoại lệ và quản lý tài nguyên 9
2.6.5 Đóng gói truy cập dữ liệu với mẫu DAO 10
2.7 Phân biệt các khái niệm dễ nhầm lẫn 10
CHƯƠNG 3. PHÂN TÍCH VÀ MINH HỌA BẰNG JAVA **11**
3.1 Môi trường và dữ liệu mẫu 11
3.2 Ví dụ 1: Truy vấn cơ bản với try-with-resources 12
3.3 Ví dụ 2: Statement và PreparedStatement trước tấn công SQL injection 14
3.4 Ví dụ 3: Tổ chức mã theo mẫu DAO, thao tác CRUD và giao dịch 16
3.5 Ví dụ 4: Gọi hàm lưu trữ bằng CallableStatement 20
3.6 Ví dụ 5: Khai thác siêu dữ liệu 22

3.7 Các lỗi thường gặp và cách xử lý 23
3.8 So sánh các cách tiếp cận 24
CHƯƠNG 4. ĐÁNH GIÁ VÀ ỨNG DỤNG 26
4.1 Ưu điểm 26
4.2 Hạn chế 26
4.3 Ứng dụng thực tế 26
4.4 Lưu ý khi áp dụng vào đồ án học phần 27
4.5 Liên hệ với nội dung học phần 27
CHƯƠNG 5. KẾT LUẬN 29
5.1 Kết quả đạt được 29
5.2 Hạn chế của báo cáo 29
5.3 Hướng phát triển 29
TÀI LIỆU THAM KHẢO 30
PHỤ LỤC A. SCRIPT TẠO CƠ SỞ DỮ LIỆU MẪU 31
PHỤ LỤC B. MÃ NGUỒN ĐẦY ĐỦ CỦA VÍ DỤ DAO 34
PHỤ LỤC C. HƯỚNG DẪN BIÊN DỊCH VÀ CHẠY 38

DANH MỤC HÌNH VẼ

Hình 2.1. Lược đồ quan hệ của CSDL mẫu gồm Student, Enrollment và Course 3

Hình 2.2. Kiến trúc phân tầng của JDBC 4

Hình 2.3. Quy trình các bước truy cập CSDL bằng JDBC 5

Hình 2.4. Quan hệ kế thừa giữa các interface câu lệnh và các phương thức tạo tương ứng của Connection 8

Hình 3.1. Cùng một đầu vào độc hại được xử lý khác nhau bởi Statement và PreparedStatement 16

Hình 3.2. Sơ đồ lớp UML của ví dụ DAO 17

Hình 3.3. Lưu đồ xử lý giao dịch trong phương thức registerCourses 19

DANH MỤC BẢNG

Bảng 2.1. Bốn interface cốt lõi của JDBC 5
Bảng 2.2. Cú pháp escape SQL dùng với CallableStatement 7
Bảng 2.3. Phân biệt các cặp khái niệm dễ nhầm lẫn 10
Bảng 3.1. Môi trường thực hiện các ví dụ 11
Bảng 3.2. Các lỗi thường gặp khi lập trình JDBC 23
Bảng 3.3. So sánh đóng tài nguyên thủ công và try-with-resources 24
Bảng 3.4. So sánh ba loại câu lệnh của JDBC 25
Bảng 4.1. Liên hệ giữa nội dung bài giảng và chủ đề JDBC 27

DANH MỤC MÃ NGUỒN

Mã nguồn 3.1. DbConfig.java – cấu hình kết nối dùng chung 12
Mã nguồn 3.2. SimpleJdbc.java – truy vấn sinh viên có họ Smith 13
Mã nguồn 3.3. FindGradeDemo.java – cùng một truy vấn viết theo hai cách 14
Mã nguồn 3.4. StudentDao.java – interface truy cập dữ liệu sinh viên 17
Mã nguồn 3.5. Phương thức registerCourses trong JdbcStudentDao.java 18
Mã nguồn 3.6. Hàm lưu trữ studentFound (MySQL 8) 20
Mã nguồn 3.7. TestCallableStatement.java – gọi hàm lưu trữ 21
Mã nguồn 3.8. MetadataDemo.java – đọc siêu dữ liệu CSDL và kết quả truy vấn 22
Mã nguồn A.1. javabook.sql – tạo CSDL, bảng, dữ liệu mẫu và hàm lưu trữ 31
Mã nguồn B.1. Student.java – lớp đối tượng miền 34
Mã nguồn B.2. JdbcStudentDao.java – hiện thực StudentDao bằng JDBC 34
Mã nguồn B.3. DaoDemo.java – chương trình chạy thử DAO 37
Mã nguồn C.1. Tạo tài khoản và cấp quyền trên CSDL javabook 38

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Tiếng Anh	Tiếng Việt
API	Application Programming Interface	Giao diện lập trình ứng dụng
CRUD	Create, Read, Update, Delete	Tạo, đọc, sửa, xóa dữ liệu
CSDL	—	Cơ sở dữ liệu
DAO	Data Access Object	Đối tượng truy cập dữ liệu
DBMS	Database Management System	Hệ quản trị cơ sở dữ liệu
DDL	Data Definition Language	Ngôn ngữ định nghĩa dữ liệu
DML	Data Manipulation Language	Ngôn ngữ thao tác dữ liệu
FK	Foreign Key	Khóa ngoại
JAR	Java Archive	Tệp đóng gói của Java
JDBC	Java Database Connectivity (tên thương mại)	API truy cập CSDL quan hệ của Java
JDK	Java Development Kit	Bộ công cụ phát triển Java
JPA	Jakarta Persistence (trước đây là Java Persistence API)	Đặc tả lưu trữ bền vững đối tượng của Java
JVM	Java Virtual Machine	Máy ảo Java
ODBC	Open Database Connectivity	Chuẩn kết nối CSDL mở
ORM	Object-Relational Mapping	Ánh xạ đối tượng – quan hệ
OWASP	Open Worldwide Application Security Project	Dự án mở về bảo mật ứng dụng
PK	Primary Key	Khóa chính
RDBMS	Relational Database Management System	Hệ quản trị CSDL quan hệ
SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc
UML	Unified Modeling Language	Ngôn ngữ mô hình hóa thống nhất
URL	Uniform Resource Locator	Chuỗi định vị tài nguyên

CHƯƠNG 1. GIỚI THIỆU

Chương này trình bày bối cảnh của chủ đề, mục tiêu, phạm vi và phương pháp thực hiện báo cáo. Đây là cơ sở để người đọc theo dõi các phân tích lý thuyết ở Chương 2 và các ví dụ minh họa ở Chương 3.

1.1 Bối cảnh và lý do chọn đề tài

Phần lớn ứng dụng phần mềm trong thực tế, từ hệ thống quản lý sinh viên, quản lý bán hàng đến các dịch vụ web, đều cần lưu trữ dữ liệu bền vững (persistent data), tức là dữ liệu vẫn còn tồn tại sau khi chương trình kết thúc. Trong đa số trường hợp, dữ liệu đó được quản lý bởi một hệ quản trị cơ sở dữ liệu quan hệ (Relational Database Management System – RDBMS) như MySQL, Oracle hay PostgreSQL, và được truy xuất bằng ngôn ngữ truy vấn có cấu trúc SQL (Structured Query Language) [1, pp. 1174–1178].

Học phần Lập trình hướng đối tượng tổ chức chương trình thành các lớp và đối tượng, dựa trên các nguyên lý trừu tượng hóa, đóng gói, kế thừa và đa hình [2]. Giữa hai thế giới này tồn tại một khoảng cách: chương trình Java làm việc với đối tượng, còn cơ sở dữ liệu (CSDL) làm việc với bảng và câu lệnh SQL. JDBC là giao diện lập trình ứng dụng (Application Programming Interface – API) chuẩn của Java giúp nối hai thế giới đó. JDBC cung cấp một giao diện thống nhất để chương trình Java gửi câu lệnh SQL, nhận kết quả và cập nhật dữ liệu mà không phụ thuộc vào một hệ quản trị cụ thể [1, p. 1189], [3].

Đề tài được lựa chọn vì ba lý do. Thứ nhất, JDBC là nền tảng mà các công nghệ ở mức cao hơn như Spring JDBC hay các bộ ánh xạ đối tượng – quan hệ (Object-Relational Mapping – ORM) như Hibernate đều xây dựng bên trên; hiểu JDBC giúp hiểu đúng những gì các công cụ đó thực hiện "phía sau". Thứ hai, bản thân thiết kế của JDBC là một ví dụ thực tế, quy mô lớn của những nguyên lý đã học trong học phần: interface, kế thừa interface, đa hình, reflection và xử lý ngoại lệ. Thứ ba, đồ án của học phần thường yêu cầu lưu trữ dữ liệu, nên kỹ năng lập trình CSDL có giá trị ứng dụng trực tiếp.

1.2 Mục tiêu tìm hiểu

Báo cáo đặt ra bốn mục tiêu cụ thể sau:

1. Trình bày kiến trúc của JDBC, vai trò của các interface cốt lõi Driver, Connection, Statement, ResultSet và quy trình các bước truy cập CSDL.
2. Phân tích ba loại câu lệnh Statement, PreparedStatement, CallableStatement cùng hai interface siêu dữ liệu DatabaseMetaData và ResultSetMetaData.
3. Chỉ ra các nguyên lý hướng đối tượng thể hiện trong thiết kế của JDBC và vận dụng chúng để tổ chức mã truy cập dữ liệu theo mẫu thiết kế DAO.
4. Xây dựng các chương trình Java biên dịch được, minh họa cả cách dùng đúng lẫn các lỗi thường gặp như SQL injection, rò rỉ tài nguyên và giao dịch dở dang.

1.3 Phạm vi và giới hạn

Báo cáo tập trung vào API JDBC lõi trong gói java.sql, dựa trên Chương 32 "Java Database Programming" của giáo trình Liang [1] và được đối chiếu với tài liệu chính thức của Oracle [3]–[6]. Hệ quản trị được chọn để minh họa là MySQL 8 với driver MySQL Connector/J 8. Dữ liệu mẫu gồm ba bảng Student, Course và Enrollment theo giáo trình [1, pp. 1175–1177].

Giáo trình được viết cho các phiên bản Java và Connector/J cũ hơn (Connector/J 5.1), do đó báo cáo cập nhật những điểm đã thay đổi, chẳng hạn tên lớp driver, cơ chế tự động nạp driver, việc loại bỏ cầu nối JDBC-ODBC và yêu cầu khai báo hàm lưu trữ trong MySQL 8. Phần giao diện đồ họa trong một số ví dụ của giáo trình được thay bằng chương trình dòng lệnh để tập trung vào logic truy cập dữ liệu. Các chủ đề như ORM, connection pool, DataSource, RowSet hay xử lý theo lô (batch) không thuộc phạm vi phân tích chi tiết và chỉ được nhắc đến ở mức định hướng trong Chương 4.

1.4 Phương pháp thực hiện

Báo cáo sử dụng phương pháp nghiên cứu tài liệu (giáo trình, bài giảng học phần, đặc tả API), kết hợp phân tích – tổng hợp và minh họa bằng mã nguồn. Toàn bộ mã nguồn được biên dịch bằng OpenJDK 21 với tùy chọn javac -Xlint:all và không phát sinh cảnh báo nào. Do môi trường soạn thảo báo cáo không có máy chủ MySQL và driver Connector/J, các chương trình chưa được chạy trên CSDL thật. Vì vậy, kết quả trong Chương 3 được ghi rõ là **kết quả dự kiến**: chúng được suy ra từ dữ liệu mẫu, và logic

của các câu truy vấn đã được đối chiếu bằng SQLite trên cùng bộ dữ liệu. Kết quả duy nhất được ghi nhận từ lần chạy thực tế là thông báo lỗi khi thiếu driver ở Mục 3.2. Hướng dẫn chạy lại chương trình trên máy cá nhân được trình bày ở Phụ lục C.

Phần còn lại của báo cáo được tổ chức như sau. Chương 2 hệ thống hóa cơ sở lý thuyết về CSDL quan hệ, kiến trúc JDBC và mối liên hệ với lập trình hướng đối tượng. Chương 3 phân tích năm ví dụ Java có độ phức tạp tăng dần, kèm bảng lỗi thường gặp và các bảng so sánh. Chương 4 đánh giá ưu điểm, hạn chế và khả năng ứng dụng. Chương 5 tổng kết kết quả và hướng phát triển.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

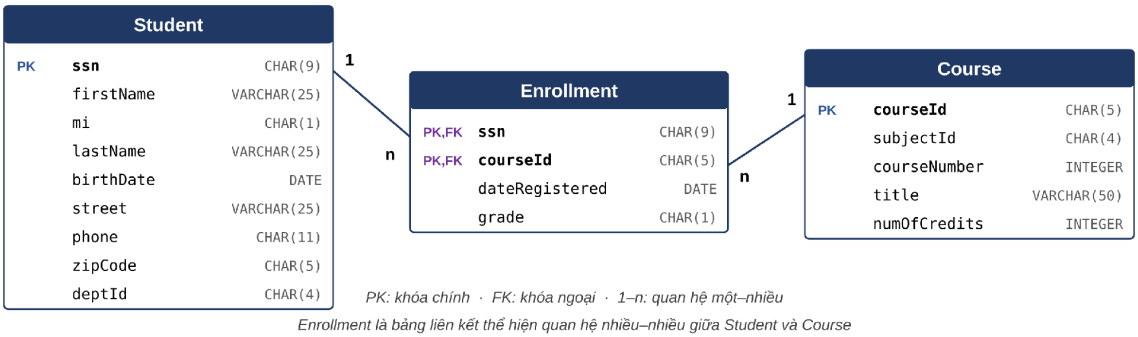
Chương này hệ thống hóa các khái niệm nền tảng theo trình tự từ dữ liệu đến chương trình: mô hình quan hệ và ngôn ngữ SQL, kiến trúc và các thành phần của JDBC, các loại câu lệnh và siêu dữ liệu. Sau đó, JDBC được nhìn lại dưới góc độ các nguyên lý hướng đối tượng. Cuối chương là bảng tổng hợp các cặp khái niệm dễ nhầm lẫn.

2.1 Cơ sở dữ liệu quan hệ và ngôn ngữ SQL

2.1.1 Mô hình dữ liệu quan hệ

Theo Liang [1, pp. 1174–1175], một hệ CSDL gồm ba thành phần: bản thân cơ sở dữ liệu, phần mềm hệ quản trị CSDL (Database Management System – DBMS) và các chương trình ứng dụng. Mô hình quan hệ biểu diễn dữ liệu dưới dạng các bảng, còn gọi là quan hệ (relation). Mỗi hàng (row, hay bộ – tuple) biểu diễn một thực thể, mỗi cột (column, hay thuộc tính – attribute) biểu diễn một đặc điểm của thực thể đó.

Tính đúng đắn của dữ liệu được bảo đảm bởi các ràng buộc toàn vẹn (integrity constraint) [1, pp. 1176–1178]. Ràng buộc miền quy định kiểu và phạm vi giá trị của cột. Ràng buộc khóa chính (primary key – PK) yêu cầu giá trị khóa là duy nhất để xác định mỗi hàng. Ràng buộc khóa ngoại (foreign key – FK) yêu cầu giá trị của cột tham chiếu phải tồn tại trong bảng được tham chiếu. Hình 2.1 thể hiện CSDL mẫu được sử dụng xuyên suốt báo cáo: Student lưu thông tin sinh viên với khóa ssn, Course lưu môn học với khóa courseId, còn Enrollment ghi nhận việc đăng ký học với khóa chính ghép (ssn, courseId), trong đó mỗi thành phần đồng thời là một khóa ngoại.



Hình 2.1. Lược đồ quan hệ của CSDL mẫu gồm Student, Enrollment và Course
Nguồn: Tác giả vẽ dựa trên dữ liệu mẫu trong [1, pp. 1175–1177]

Nhờ ràng buộc khóa ngoại, DBMS sẽ từ chối một bản ghi Enrollment tham chiếu đến một môn học không tồn tại. Tính chất này được khai thác trong ví dụ về giao dịch

ở Mục 3.4. Cần lưu ý rằng một số kết quả minh họa trong giáo trình không hoàn toàn khớp với bảng dữ liệu được công bố trong cùng chương; báo cáo thống nhất sử dụng dữ liệu ở bảng Student của giáo trình (Hình 32.4 trong [1]) và tự suy ra mọi kết quả từ bộ dữ liệu đó.

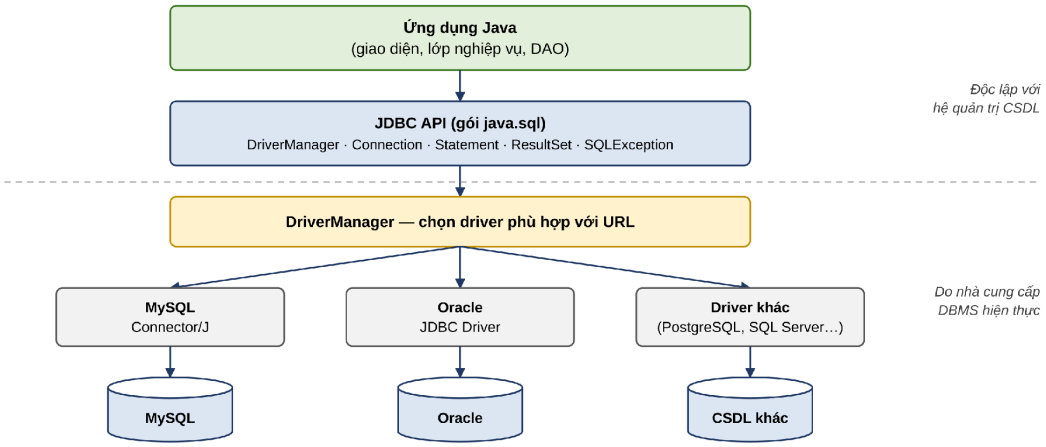
2.1.2 Ngôn ngữ truy vấn SQL

SQL là ngôn ngữ chuẩn để định nghĩa và thao tác dữ liệu trong các hệ quan hệ [1, pp. 1178–1188]. Các câu lệnh SQL thường được chia thành hai nhóm. Nhóm ngôn ngữ định nghĩa dữ liệu (Data Definition Language – DDL) gồm các lệnh như CREATE TABLE và DROP TABLE để tạo hoặc xóa cấu trúc. Nhóm ngôn ngữ thao tác dữ liệu (Data Manipulation Language – DML) gồm SELECT để truy vấn, cùng INSERT, UPDATE, DELETE để cập nhật dữ liệu. Ví dụ, câu lệnh SELECT firstName, lastName FROM Student WHERE deptId = 'BIOL' trả về họ tên các sinh viên thuộc khoa Sinh học.

Sự phân biệt hai nhóm câu lệnh này có ý nghĩa trực tiếp đối với JDBC, vì nó quyết định phương thức được dùng để thực thi: câu SELECT trả về một tập kết quả dạng bảng, trong khi các câu cập nhật và DDL chỉ trả về số dòng bị ảnh hưởng (Mục 2.3.2).

2.2 Kiến trúc JDBC

JDBC là tên thương mại của API truy cập CSDL quan hệ trong Java. Tên này thường được hiểu là Java Database Connectivity, tuy giáo trình lưu ý rằng về mặt chính thức JDBC không phải là một từ viết tắt [1, p. 1189]. Kiến trúc JDBC gồm hai tầng tách biệt (Hình 2.2). Tầng trên là JDBC API trong gói java.sql, chủ yếu là các interface do Java định nghĩa. Tầng dưới là JDBC driver, tức các lớp cụ thể do nhà cung cấp DBMS viết để hiện thực các interface đó và giao tiếp với DBMS bằng giao thức riêng [1, pp. 1189–1190].



Hình 2.2. Kiến trúc phân tầng của JDBC

Nguồn: Tác giả vẽ lại, có cập nhật, dựa trên Hình 32.20 trong [1] và [6]

Vì ứng dụng chỉ lập trình với tầng trên, việc chuyển từ MySQL sang Oracle về nguyên tắc chỉ cần đổi driver và chuỗi kết nối, với điều kiện các câu SQL không dùng cú pháp riêng của từng hệ quản trị. Một điểm cần cập nhật so với giáo trình: Hình 32.20 trong [1] vẫn trình bày cầu nối JDBC-ODBC để truy cập Microsoft Access, nhưng thành phần này đã bị loại khỏi JDK kể từ Java 8 [10]. Hiện nay, mỗi DBMS được truy cập trực tiếp qua driver JDBC riêng; với MySQL Connector/J 8, lớp driver có tên com.mysql.cj.jdbc.Driver [8].

2.3 Các interface cốt lõi và quy trình truy cập CSDL

Giáo trình xác định bốn interface then chốt mà mọi ứng dụng CSDL bằng Java đều cần đến là Driver, Connection, Statement và ResultSet [1, p. 1190]. Bảng 2.1 tóm tắt vai trò, cách tạo và các phương thức tiêu biểu của từng interface.

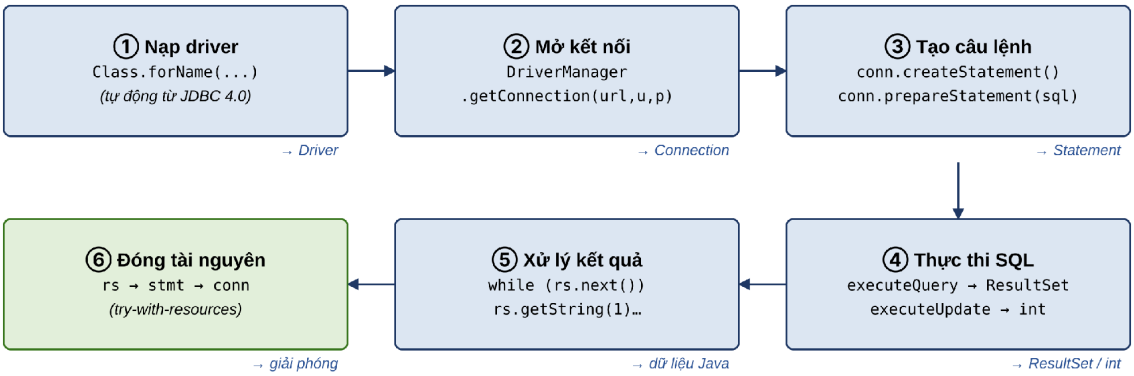
Bảng 2.1. Bốn interface cốt lõi của JDBC

Interface	Vai trò	Cách có được đối tượng	Phương thức tiêu biểu
Driver	Đại diện cho driver của một DBMS; biết cách mở kết nối tới URL phù hợp	Lớp driver tự đăng ký với DriverManager khi được nạp	connect(url, info)
Connection	Một phiên làm việc với CSDL; quản lý giao dịch; tạo các đối tượng câu lệnh	DriverManager.getConnection(url, user, pw)	createStatement(), setAutoCommit(), commit(), rollback()
Statement	Gửi câu lệnh SQL tới DBMS và nhận kết quả	conn.createStatement()	executeQuery(), executeUpdate(), execute()

Interface	Vai trò	Cách có được đối tượng	Phương thức tiêu biểu
ResultSet	Bảng kết quả của truy vấn, được duyệt bằng con trỏ	stmt.executeQuery(sql)	next(), getString(), getInt()

Nguồn: Tác giả tổng hợp từ [1, pp. 1190–1192] và [6]

Các interface này phối hợp với nhau theo một quy trình gồm năm bước được giáo trình mô tả [1, pp. 1190–1193]; báo cáo bổ sung bước thứ sáu là đóng tài nguyên, vốn rất quan trọng trong thực tế (Hình 2.3). Mỗi bước tạo ra đối tượng làm đầu vào cho bước kế tiếp: driver giúp tạo Connection, Connection tạo Statement, Statement thực thi câu lệnh để tạo ResultSet.



Hình 2.3. Quy trình các bước truy cập CSDL bằng JDBC

Nguồn: Tác giả tổng hợp từ [1, pp. 1190–1193] và [3]

2.3.1 Nạp driver và mở kết nối

Trong giáo trình, bước đầu tiên là nạp lớp driver bằng Class.forName("tên lớp") [1, pp. 1190–1191]. Lời gọi này nạp lớp theo tên tại thời điểm chạy; khi lớp driver được khởi tạo, nó tự đăng ký một thể hiện của mình với DriverManager. Từ JDBC 4.0 (Java 6), DriverManager có thể tự tìm các driver thông qua cơ chế service provider khi file JAR của driver có mặt trên class path [1, p. 1191], [6], vì vậy bước này thường có thể bỏ qua. Nếu vẫn dùng Class.forName, cần chú ý tên lớp: giáo trình dùng com.mysql.jdbc.Driver của Connector/J 5.1, trong khi Connector/J 8 dùng com.mysql.cj.jdbc.Driver [8].

Kết nối được mở bằng DriverManager.getConnection(url, user, password). Chuỗi kết nối (URL) có dạng jdbc:subprotocol:subname; với MySQL là jdbc:mysql://hostname:port/dbname [1, p. 1191]. DriverManager duyệt các driver đã đăng ký để tìm driver chấp nhận URL đó. Nếu không tìm được, phương thức ném

SQLException với thông báo "No suitable driver found", lỗi thường gặp nhất khi quên đưa file JAR của driver vào class path (Mục 3.7).

2.3.2 Thực thi câu lệnh và duyệt ResultSet

Đối tượng Statement được tạo từ Connection bằng createStatement(). Phương thức executeQuery(sql) dùng cho câu SELECT và trả về một ResultSet; executeUpdate(sql) dùng cho INSERT, UPDATE, DELETE hoặc các câu DDL và trả về một số nguyên là số dòng bị ảnh hưởng; còn execute(sql) dùng khi chưa biết trước loại câu lệnh, trả về giá trị boolean cho biết kết quả đầu tiên có phải là ResultSet hay không [1, pp. 1191–1192], [6].

ResultSet duy trì một bảng kết quả cùng một con trỏ (cursor). Ban đầu con trỏ nằm **trước dòng đầu tiên**, vì vậy phải gọi next() trước khi đọc dữ liệu; next() dịch con trỏ xuống một dòng và trả về false khi không còn dòng nào [1, pp. 1192–1193], [6]. Các phương thức getXxx nhận tên cột hoặc chỉ số cột, trong đó chỉ số **bắt đầu từ 1** chứ không phải từ 0 như chỉ số mảng trong Java. Dùng sai phương thức thực thi, chẳng hạn gọi executeQuery cho câu INSERT, sẽ gây ra SQLException.

2.3.3 Giao dịch và chế độ tự động xác nhận

Giao dịch (transaction) là một nhóm thao tác phải cùng thành công hoặc cùng thất bại. Mặc định, một Connection hoạt động ở chế độ tự động xác nhận (auto-commit): mỗi câu lệnh là một giao dịch riêng và được xác nhận ngay khi thực thi xong [4], [6]. Khi cần nhiều câu lệnh tạo thành một đơn vị, lập trình viên tắt chế độ này bằng setAutoCommit(false), gọi commit() khi mọi bước thành công và rollback() khi có lỗi [1, p. 1194], [4].

Đặc tả API còn nêu một chi tiết tinh tế nhưng quan trọng: nếu setAutoCommit được gọi trong khi một giao dịch đang diễn ra và làm thay đổi chế độ, giao dịch hiện tại sẽ được **xác nhận (commit)** [6]. Do đó, việc khôi phục chế độ auto-commit trong khối finally chỉ an toàn khi nhánh xử lý lỗi đã gọi rollback() trước đó. Điểm này được phân tích cụ thể ở Mục 3.4.

2.4 PreparedStatement và CallableStatement

2.4.1 PreparedStatement

PreparedStatement là interface mở rộng Statement, dùng để thực thi câu lệnh SQL được biên dịch trước (precompiled), có thể chứa các tham số được đánh dấu bằng dấu ?

[1, p. 1197]. Đối tượng được tạo bằng conn.prepareStatement(sql), nên câu SQL gắn với đối tượng ngay từ lúc tạo. Giá trị tham số được gán bằng setXxx(chỉ số, giá trị) với chỉ số bắt đầu từ 1; sau đó gọi executeQuery() hoặc executeUpdate() **không có đối số**.

Cách tiếp cận này mang lại ba lợi ích. Thứ nhất là hiệu năng khi thực thi lặp lại, dù mức độ cải thiện phụ thuộc việc driver có thực sự biên dịch trước hay không [6]. Thứ hai là mã nguồn rõ ràng hơn so với ghép chuỗi. Thứ ba, và quan trọng nhất, là an toàn: giá trị tham số luôn được xử lý như dữ liệu và không thể làm thay đổi cấu trúc câu lệnh. Đây là biện pháp phòng chống tấn công chèn mã SQL (SQL injection) được OWASP xếp ở vị trí ưu tiên hàng đầu [11].

2.4.2 CallableStatement

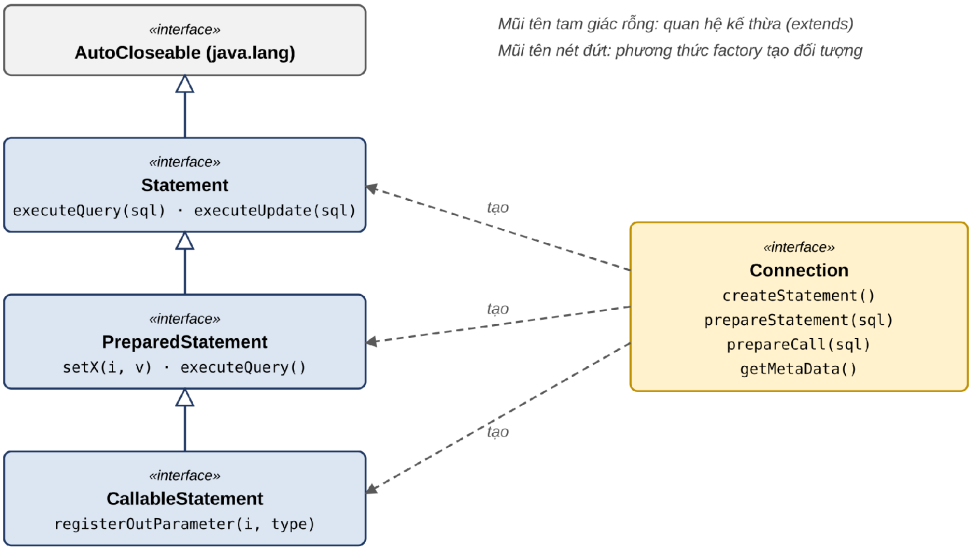
CallableStatement mở rộng PreparedStatement để gọi các thủ tục (procedure) và hàm (function) được lưu trữ trên DBMS [1, pp. 1199–1201]. Tham số của thủ tục có ba loại: tham số IN nhận giá trị truyền vào; tham số OUT trả giá trị ra sau khi thủ tục kết thúc; tham số IN OUT vừa nhận vào vừa trả ra. Câu gọi được viết bằng cú pháp escape SQL của JDBC; driver sẽ dịch cú pháp này sang cú pháp riêng của DBMS (Bảng 2.2). Các tham số OUT và IN OUT phải được đăng ký kiểu bằng registerOutParameter(chỉ số, kiểu) trước khi thực thi, sau đó đọc giá trị bằng getXxx sau khi thực thi.

Bảng 2.2. Cú pháp escape SQL dùng với CallableStatement

Cú pháp	Ý nghĩa
{call tenThuTuc(?, ?)}	Gọi thủ tục lưu trữ có tham số
{call tenThuTuc}	Gọi thủ tục lưu trữ không có tham số
{? = call tenHam(?, ?)}	Gọi hàm lưu trữ; tham số thứ nhất nhận giá trị trả về của hàm

Nguồn: [1, p. 1200]

Như Hình 2.4 cho thấy, ba loại câu lệnh tạo thành một chuỗi kế thừa interface, và cả ba đều được tạo ra bởi các phương thức tương ứng của Connection. Vì CallableStatement kế thừa PreparedStatement, nó có sẵn các phương thức setXxx để gán tham số IN và bổ sung các phương thức làm việc với tham số OUT.



Hình 2.4. Quan hệ kế thừa giữa các interface câu lệnh và các phương thức tạo tương ứng của Connection

Nguồn: Tác giả vẽ dựa trên [1] và [6]

2.5 Siêu dữ liệu

Siêu dữ liệu (metadata) là dữ liệu mô tả dữ liệu. JDBC cung cấp hai interface cho mục đích này [1, pp. 1202–1205]. DatabaseMetaData mô tả toàn bộ CSDL: tên và phiên bản DBMS, tên và phiên bản driver, danh sách các bảng; đối tượng này được lấy bằng conn.getMetaData(). ResultSetMetaData mô tả cấu trúc của một ResultSet cụ thể như số cột, tên và kiểu của từng cột; đối tượng này được lấy bằng rs.getMetaData().

Phương thức getTables(catalog, schemaPattern, tableNamePattern, types) của DatabaseMetaData trả về chính một ResultSet, trong đó mỗi dòng mô tả một bảng và cột TABLE_NAME chứa tên bảng [1, p. 1204]. Giá trị thực tiễn của siêu dữ liệu là cho phép viết mã tổng quát: một phương thức in bảng có thể hoạt động với mọi câu SELECT mà không cần biết trước tên và số lượng cột (Mục 3.6). Giáo trình truyền null cho tham số catalog; với Connector/J 8, cách này có thể làm kết quả bao gồm bảng của các CSDL khác trên cùng máy chủ, nên ví dụ trong báo cáo truyền conn.getCatalog() để giới hạn trong CSDL đang kết nối.

2.6 JDBC dưới góc nhìn lập trình hướng đối tượng

JDBC không chỉ là một thư viện truy cập dữ liệu; thiết kế của nó là minh họa thực tế cho nhiều nguyên lý đã học trong học phần [2]. Mục này phân tích năm khía cạnh tiêu biểu.

2.6.1 Trừu tượng hóa và đa hình qua interface

Hầu hết thành phần của gói java.sql là interface. Trong câu lệnh Connection conn = DriverManager.getConnection(url, user, pw), biến conn có kiểu khai báo là interface Connection, nhưng đối tượng thực sự thuộc về một lớp cụ thể nằm trong driver. Khi gọi conn.createStatement(), lời gọi được liên kết động (dynamic binding) tới hiện thực của lớp đó; đây chính là đa hình. Nhờ vậy, mã ứng dụng không phụ thuộc vào lớp cụ thể của nhà cung cấp. Tư tưởng "tách interface khỏi hiện thực" này cũng là nguyên tắc tổ chức của thư viện Collections, nơi List được tách khỏi ArrayList và LinkedList [2, Ch. 9]. Bên cạnh đó, Connection đóng vai trò một "nhà máy" (factory): chương trình không dùng toán tử new để tạo Statement mà yêu cầu Connection tạo, nhờ đó driver có thể trả về đối tượng phù hợp với phiên kết nối.

2.6.2 Kế thừa giữa các interface

Một interface có thể kế thừa interface khác bằng từ khóa extends [2, Ch. 6]. Trong JDBC, PreparedStatement kế thừa Statement và CallableStatement kế thừa PreparedStatement (Hình 2.4). Do đó, một đối tượng CallableStatement có đầy đủ khả năng của PreparedStatement và bổ sung các phương thức như registerOutParameter; ở chiều ngược lại, một biến kiểu Statement có thể tham chiếu tới một đối tượng PreparedStatement (upcasting). Chuỗi kế thừa này kéo dài tới interface AutoCloseable, nhờ đó Connection, Statement và ResultSet có thể được dùng trong câu lệnh try-with-resources.

2.6.3 Reflection và cơ chế tự động phát hiện driver

Class.forName(tenLop) thuộc nhóm API reflection, cho phép nạp và khởi tạo một lớp từ chuỗi tên tại thời điểm chạy [2, Ch. 5]. Nhờ đó, mã nguồn ứng dụng không cần tham chiếu tĩnh tới lớp driver, và tên lớp thậm chí có thể được đọc từ tệp cấu hình. Từ JDBC 4.0, việc nạp này được tự động hóa: DriverManager dùng cơ chế service provider để tìm các driver được khai báo trong các file JAR trên class path [6]. Điều này giải thích vì sao chương trình biên dịch được mà không cần driver, nhưng khi chạy thì driver phải nằm trong class path [2, Ch. 4]. Đây là biểu hiện của nguyên tắc phụ thuộc vào trừu tượng thay vì phụ thuộc vào hiện thực cụ thể.

2.6.4 Xử lý ngoại lệ và quản lý tài nguyên

Hầu hết phương thức JDBC khai báo throws SQLException. Đây là ngoại lệ được kiểm tra (checked exception), nên trình biên dịch buộc lập trình viên phải bắt hoặc khai

báo ném tiếp [2, Ch. 7]. Ngoài thông báo lỗi, SQLException còn cung cấp mã trạng thái chuẩn qua getSQLState() và mã lỗi của nhà cung cấp qua getErrorCode(). Mặt khác, các đối tượng Connection, Statement và ResultSet giữ tài nguyên nằm ngoài JVM như kết nối mạng và con trỏ phía máy chủ, mà bộ thu gom rác không giải phóng kịp thời, nên chúng phải được đóng tường minh.

Câu lệnh try-with-resources (Java 7) tự động gọi close() theo thứ tự ngược với thứ tự khai báo, kể cả khi có ngoại lệ xảy ra. Nếu việc đóng tài nguyên cũng phát sinh ngoại lệ, ngoại lệ đó được lưu lại dưới dạng ngoại lệ bị nén (suppressed exception) thay vì che mất ngoại lệ gốc [5], [2, Ch. 7]. Giáo trình [1] đóng kết nối thủ công bằng connection.close(); báo cáo áp dụng try-with-resources cho tất cả các ví dụ.

2.6.5 Đóng gói truy cập dữ liệu với mẫu DAO

Mẫu thiết kế Đối tượng truy cập dữ liệu (Data Access Object – DAO) gom toàn bộ mã SQL liên quan đến một thực thể vào một lớp riêng, ẩn sau một interface. Các phần khác của chương trình chỉ thấy những phương thức như findBySsn hay insert và làm việc với đối tượng miền như Student, không thấy câu SQL, Connection hay ResultSet. Đây là cách áp dụng nguyên lý đóng gói và các gợi ý thiết kế lớp trong bài giảng: dữ liệu để ở mức private và mỗi lớp đảm nhận một trách nhiệm rõ ràng [2, Ch. 4]. DAO không thuộc nội dung Chương 32 của [1]; báo cáo đưa vào như một cách vận dụng kiến thức hướng đối tượng vào lập trình JDBC (Mục 3.4).

2.7 Phân biệt các khái niệm dễ nhầm lẫn

Bảng 2.3 tổng hợp các cặp khái niệm thường bị nhầm lẫn khi mới học JDBC, nhằm hệ thống lại nội dung của chương trước khi chuyển sang phần minh họa.

Bảng 2.3. Phân biệt các cặp khái niệm dễ nhầm lẫn

Cặp khái niệm	Điểm phân biệt
JDBC API và JDBC driver	API là tập interface chuẩn trong java.sql do Java định nghĩa; driver là các lớp cụ thể hiện thực API, do nhà cung cấp DBMS viết và phân phối trong file JAR riêng.
Driver và DriverManager	Driver là interface được mỗi driver hiện thực; DriverManager là lớp của Java quản lý các driver đã đăng ký và tạo kết nối.
executeQuery và executeUpdate	executeQuery dùng cho SELECT và trả về ResultSet; executeUpdate dùng cho INSERT/UPDATE/DELETE/DDL và trả về số dòng bị ảnh hưởng.

Cặp khái niệm	Điểm phân biệt
Statement và PreparedStatement	Statement nhận câu SQL lúc thực thi; PreparedStatement gắn câu SQL có tham số ? lúc tạo, được biên dịch trước và chống SQL injection.
PreparedStatement và CallableStatement	PreparedStatement thực thi câu SQL thông thường; CallableStatement gọi thủ tục/hàm lưu trữ và hỗ trợ tham số OUT, IN OUT.
DatabaseMetaData và ResultSetMetaData	Loại thứ nhất mô tả toàn bộ CSDL (conn.getMetaData()); loại thứ hai mô tả các cột của một ResultSet (rs.getMetaData()).
Auto-commit và commit()	Auto-commit: mỗi câu lệnh tự là một giao dịch và được xác nhận ngay; commit(): xác nhận thủ công một nhóm câu lệnh sau khi đã tắt auto-commit.

Nguồn: Tác giả tổng hợp từ [1] và [6]

CHƯƠNG 3. PHÂN TÍCH VÀ MINH HỌA BẰNG JAVA

Chương này hiện thực hóa các khái niệm của Chương 2 bằng năm ví dụ có độ phức tạp tăng dần: truy vấn cơ bản, so sánh Statement với PreparedStatement trước tấn công SQL injection, tổ chức mã theo mẫu DAO kèm giao dịch, gọi hàm lưu trữ và khai thác siêu dữ liệu. Mỗi ví dụ gồm mã nguồn, giải thích theo số dòng, kết quả và phân tích. Phần cuối chương tổng hợp các lỗi thường gặp và so sánh các cách tiếp cận.

Về tính xác thực của kết quả: toàn bộ mã nguồn đã được biên dịch thành công bằng OpenJDK 21 với tùy chọn -Xlint:all và không có cảnh báo. Như đã nêu ở Mục 1.4, các chương trình chưa được chạy trên máy chủ MySQL; các khối có tiêu đề "Kết quả dự kiến" được suy ra từ dữ liệu mẫu, trong đó logic truy vấn của các Ví dụ 1, 2, 3 và phần dữ liệu của Ví dụ 5 đã được đối chiếu bằng SQLite. Khối có tiêu đề "Kết quả chạy thực tế" là đầu ra thu được khi chạy chương trình.

3.1 Môi trường và dữ liệu mẫu

Bảng 3.1 liệt kê môi trường cần thiết để chạy các ví dụ. Script tạo CSDL javabook cùng dữ liệu mẫu được trình bày đầy đủ ở Phụ lục A.

Bảng 3.1. Môi trường thực hiện các ví dụ

Thành phần	Phiên bản / ghi chú
Bộ công cụ Java (JDK)	OpenJDK 21 (dùng để biên dịch); mã nguồn yêu cầu Java 9 trở lên do dùng List.of
Hệ quản trị CSDL	MySQL Server 8.x
JDBC driver	MySQL Connector/J 8.x (tệp mysql-connector-j-8.x.x.jar)
Cơ sở dữ liệu	javabook gồm Student (11 dòng), Course (8 dòng), Enrollment (19 dòng) theo [1, pp. 1175–1177]
Tài khoản kết nối	scott/tiger như giáo trình; có thể thay bằng -Ddb.user, -Ddb.password
Kiểm chứng logic SQL	SQLite 3 (mô-đun sqlite3 của Python) trên cùng bộ dữ liệu

Nguồn: Tác giả

Lớp DbConfig (Mã nguồn 3.1) tập trung thông tin kết nối vào một nơi để các ví dụ dùng chung. Lớp được khai báo final với constructor private (dòng 17), tức là một lớp tiện ích chỉ chứa thành viên static và không cho phép tạo đối tượng. Các hằng URL, USER, PASSWORD được đọc từ thuộc tính hệ thống (system property) với giá trị mặc

định (dòng 10–15), cho phép thay đổi cấu hình lúc chạy mà không phải sửa và biên dịch lại mã. Phương thức getConnection() (dòng 19–21) không gọi Class.forName vì dựa vào cơ chế tự động phát hiện driver của JDBC 4.0 (Mục 2.6.3).

Mã nguồn 3.1. DbConfig.java – cấu hình kết nối dùng chung

```
 1  import java.sql.Connection;
 2  import java.sql.DriverManager;
 3  import java.sql.SQLException;
 4
 5  /**
 6   * Tập trung thông tin kết nối. Có thể ghi đè khi chạy bằng
 7   * -Ddb.url=..., -Ddb.user=..., -Ddb.password=...
 8   */
 9  public final class DbConfig {
10    private static final String URL = System.getProperty(
11        "db.url", "jdbc:mysql://localhost:3306/javabook");
12    private static final String USER =
13        System.getProperty("db.user", "scott");
14    private static final String PASSWORD =
15        System.getProperty("db.password", "tiger");
16
17    private DbConfig() { }   // lớp tiện ích: không cho tạo đối tượng
18
19    public static Connection getConnection() throws SQLException {
20      return DriverManager.getConnection(URL, USER, PASSWORD);
21    }
22  }
```

3.2 Ví dụ 1: Truy vấn cơ bản với try-with-resources

Ví dụ này tương ứng với chương trình SimpleJdbc (Listing 32.1) trong [1, p. 1193], được viết lại với hai thay đổi: dùng try-with-resources thay cho việc đóng kết nối thủ công, và bắt SQLException tại chỗ để in thông tin chẩn đoán.

Mã nguồn 3.2. SimpleJdbc.java – truy vấn sinh viên có họ Smith

```
 1  import java.sql.Connection;
 2  import java.sql.ResultSet;
 3  import java.sql.SQLException;
 4  import java.sql.Statement;
 5
 6  public class SimpleJdbc {
 7    public static void main(String[] args) {
 8      String sql = "SELECT firstName, mi, lastName FROM Student "
 9              + "WHERE lastName = 'Smith'";
10      // Bước 2-4: kết nối, tạo Statement, thực thi truy vấn.
11      // Ba tài nguyên tự động đóng theo thứ tự ngược: rs, stmt, conn.
12      try (Connection conn = DbConfig.getConnection();
13           Statement stmt = conn.createStatement();
14           ResultSet rs = stmt.executeQuery(sql)) {
15        System.out.println("Database connected");
16        while (rs.next()) {              // Bước 5: duyệt từng dòng
17          System.out.println(rs.getString(1) + "\t"
18              + rs.getString(2) + "\t" + rs.getString(3));
19        }
20      } catch (SQLException ex) {
21        System.err.println("Loi CSDL: " + ex.getMessage()
22            + " (SQLState=" + ex.getSQLState() + ")");
23      }
24    }
25  }
```

Dòng 12–14 khai báo ba tài nguyên trong phần đầu của khối try. Chúng được tạo theo thứ tự Connection → Statement → ResultSet và được đóng theo thứ tự ngược lại khi khối try kết thúc, dù kết thúc bình thường hay do ngoại lệ [5]. Dòng 16 là vòng lặp chuẩn để duyệt ResultSet: next() vừa dịch con trỏ vừa cho biết còn dòng dữ liệu hay không. Dòng 17–18 đọc cột theo chỉ số 1, 2, 3; các chỉ số này tương ứng với thứ tự cột trong mệnh đề SELECT, không phải thứ tự cột trong bảng. Dòng 21–22 in thêm SQLState, một mã chuẩn hóa giúp phân loại lỗi.

Kết quả dự kiến (với CSDL mẫu ở Phụ lục A):

```
Database connected
Jacob   R   Smith
George  K   Smith
Jean    K   Smith
Josh    R   Smith
```

Bốn sinh viên trên khớp với kết quả của cùng câu truy vấn khi chạy trên SQLite. Vì câu SELECT không có mệnh đề ORDER BY, thứ tự các dòng không được chuẩn SQL bảo đảm và có thể khác nhau giữa các DBMS; nếu cần thứ tự xác định, phải bổ sung ORDER BY. Khi chạy chương trình trong môi trường chưa có driver trong class path, kết quả thu được như sau:

Kết quả chạy thực tế (chưa có driver trong class path):

```
Loi CSDL: No suitable driver found for jdbc:mysql://localhost:3306/javabook
      (SQLState=08001)
```

Kết quả này xác nhận hai điều. Một là chương trình biên dịch và khởi chạy được dù thiếu driver, vì mã chỉ phụ thuộc vào các interface trong java.sql; lỗi chỉ xuất hiện khi DriverManager không tìm được driver nhận URL ở thời điểm chạy. Hai là nhờ try-with-resources và khối catch, chương trình kết thúc có kiểm soát với thông báo rõ ràng thay vì in toàn bộ vết ngăn xếp (stack trace). Mã SQLState 08001 thuộc lớp 08 – nhóm lỗi kết nối. Thông báo trong chương trình được viết tiếng Việt không dấu để tránh lỗi hiển thị bảng mã trên một số cửa sổ dòng lệnh.

3.3 Ví dụ 2: Statement và PreparedStatement trước tấn công SQL injection

Giáo trình minh họa lỗ hổng của việc ghép chuỗi bằng chương trình tra điểm theo mã sinh viên và mã môn: nếu nhập chuỗi 1' or true or '1 vào ô mã sinh viên, chương trình sẽ trả về dữ liệu không mong muốn [1, p. 1196]; sau đó phiên bản dùng PreparedStatement được giới thiệu như giải pháp [1, p. 1198]. Ví dụ 2 đặt hai cách hiện thực cạnh nhau để làm rõ sự khác biệt. Chuỗi tấn công được chọn là ' OR '1'='1, chỉ dùng phép so sánh chuỗi chuẩn nên có tác dụng trên nhiều DBMS, trong khi chuỗi của giáo trình dựa vào việc MySQL coi '1' là giá trị đúng.

Mã nguồn 3.3. FindGradeDemo.java – cùng một truy vấn viết theo hai cách

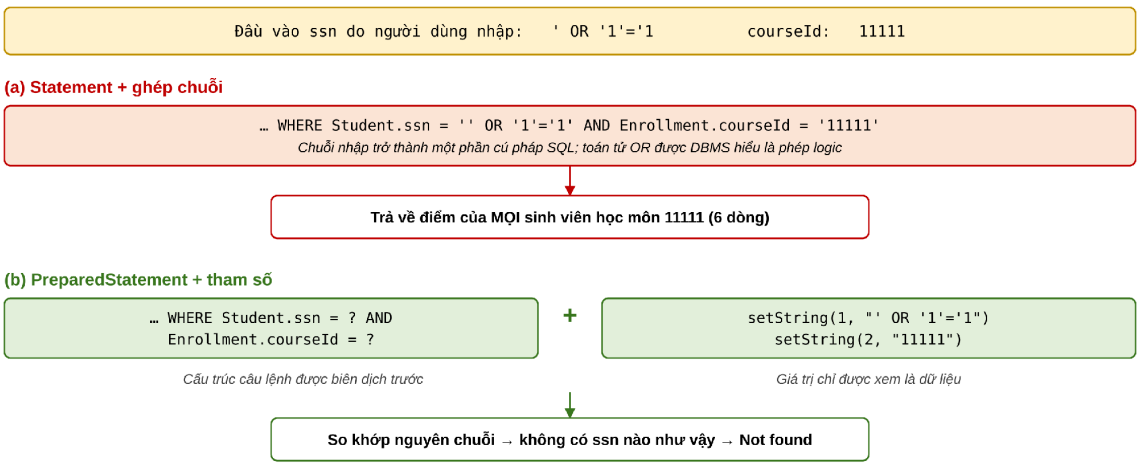
```
 1  import java.sql.Connection;
 2  import java.sql.PreparedStatement;
 3  import java.sql.ResultSet;
 4  import java.sql.SQLException;
 5  import java.sql.Statement;
 6
 7  public class FindGradeDemo {
 8    private static final String BASE =
 9        "SELECT firstName, lastName, title, grade FROM Student "
10      + "JOIN Enrollment ON Student.ssn = Enrollment.ssn "
11      + "JOIN Course ON Enrollment.courseId = Course.courseId ";
12
13    /** KHÔNG an toàn: ghép trực tiếp dữ liệu nhập vào câu SQL. */
14    static void findGradeUnsafe(Connection conn, String ssn,
15        String courseId) throws SQLException {
16      String sql = BASE + "WHERE Student.ssn = '" + ssn + "' "
17                + "AND Enrollment.courseId = '" + courseId + "'";
18      try (Statement stmt = conn.createStatement();
19           ResultSet rs = stmt.executeQuery(sql)) {
20        print("Statement", rs);
21      }
22    }
23
24    /** An toàn: dữ liệu được gửi tách biệt qua tham số '?'. */
25    static void findGradeSafe(Connection conn, String ssn,
26        String courseId) throws SQLException {
```

```
27    String sql = BASE
28        + "WHERE Student.ssn = ? AND Enrollment.courseId = ?";
29    try (PreparedStatement ps = conn.prepareStatement(sql)) {
30      ps.setString(1, ssn);          // chỉ số tham số bắt đầu từ 1
31      ps.setString(2, courseId);
32      try (ResultSet rs = ps.executeQuery()) {  // không truyền SQL
33        print("PreparedStatement", rs);
34      }
35    }
36  }
37
38  private static void print(String tag, ResultSet rs)
39      throws SQLException {
40    int count = 0;
41    while (rs.next()) {
42      count++;
43      System.out.println("[" + tag + "] "
44          + rs.getString("firstName") + " "
45          + rs.getString("lastName") + " | "
46          + rs.getString("title") + " | " + rs.getString("grade"));
47    }
48    if (count == 0) {
49      System.out.println("[" + tag + "] Not found");
50    }
51  }
52
53  public static void main(String[] args) {
54    String attack = "' OR '1'='1";
55    try (Connection conn = DbConfig.getConnection()) {
56      findGradeUnsafe(conn, "444111110", "11111");  // hợp lệ
57      findGradeSafe(conn, "444111110", "11111");
58      findGradeUnsafe(conn, attack, "11111");       // độc hại
59      findGradeSafe(conn, attack, "11111");
60    } catch (SQLException ex) {
61      System.err.println("Loi CSDL: " + ex.getMessage());
62    }
63  }
64 }
```

Phương thức findGradeUnsafe (dòng 14–22) dựng câu SQL bằng cách ghép chuỗi (dòng 16–17): giá trị ssn do người dùng cung cấp được chèn thẳng vào giữa hai dấu nháy đơn. Phương thức findGradeSafe (dòng 25–36) dùng câu SQL cố định với hai dấu ? (dòng 28), gán giá trị bằng setString (dòng 30–31) và gọi executeQuery() không đối số (dòng 32). Cả hai dùng chung phương thức print (dòng 38–51), đọc cột theo tên thay vì chỉ số để mã dễ đọc hơn.

Với đầu vào hợp lệ (dòng 56–57), hai cách cho cùng một kết quả. Khác biệt xuất hiện khi đầu vào là chuỗi độc hại (dòng 54, 58–59), được minh họa ở Hình 3.1. Khi ghép chuỗi, dấu nháy đầu tiên của đầu vào đóng chuỗi ssn sớm, phần còn lại trở thành mã SQL. Vì toán tử AND có độ ưu tiên cao hơn OR, điều kiện được DBMS hiểu là ssn = '' OR ('1'='1' AND courseId = '11111'). Vế phải đúng với mọi lượt đăng ký môn 11111, nên truy vấn trả về điểm của tất cả sinh viên học môn này: một sự rò rỉ dữ liệu. Với

PreparedStatement, toàn bộ chuỗi được gửi như một giá trị; DBMS so sánh ssn với nguyên văn chuỗi đó và không tìm thấy dòng nào.



Hình 3.1. Cùng một đầu vào độc hại được xử lý khác nhau bởi Statement và PreparedStatement

Nguồn: Tác giả minh họa dựa trên ví dụ ở Mục 3.3 và [11]

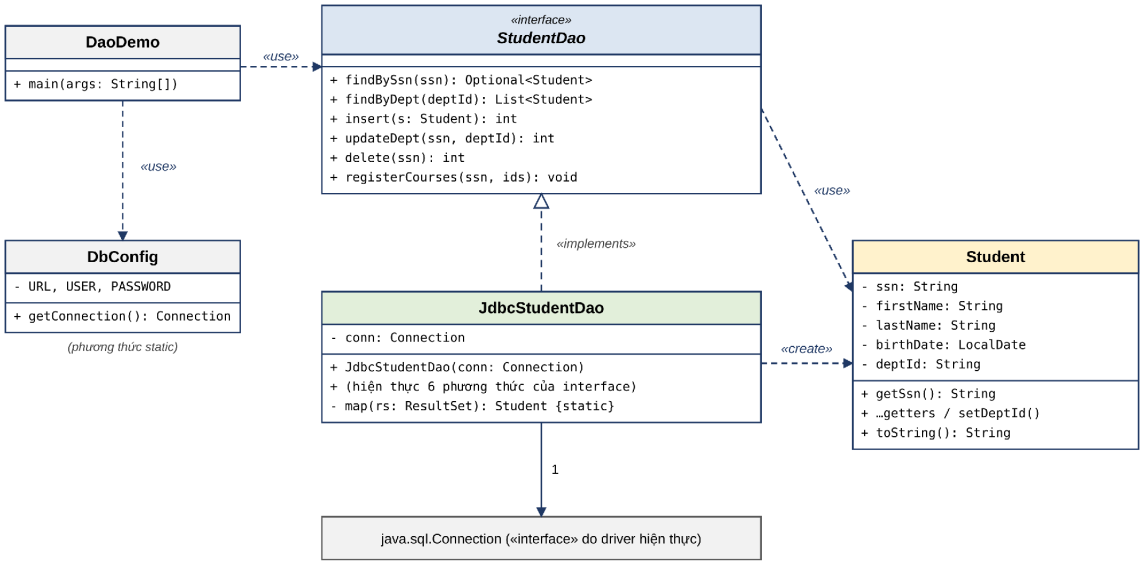
Kết quả dự kiến:

```
[Statement] Jacob Smith | Introduction to Java I | A
[PreparedStatement] Jacob Smith | Introduction to Java I | A
[Statement] Jacob Smith | Introduction to Java I | A
[Statement] John Stevenson | Introduction to Java I | D
[Statement] Frank Jones | Introduction to Java I | A
[Statement] Josh Smith | Introduction to Java I | D
[Statement] Joy Kennedy | Introduction to Java I | D
[Statement] Toni Peterson | Introduction to Java I | A
[PreparedStatement] Not found
```

Khi đối chiếu trên SQLite, lời gọi thứ ba trả về đúng sáu dòng (sáu sinh viên đăng ký môn 11111 trong bảng Enrollment) và lời gọi thứ tư trả về không dòng nào; thứ tự các dòng có thể khác nhau tùy DBMS. Cần lưu ý giới hạn của PreparedStatement: dấu ? chỉ thay thế được **giá trị**, không thay được tên bảng, tên cột hay từ khóa như chiều sắp xếp trong ORDER BY. Trong những trường hợp đó, đầu vào phải được kiểm tra theo danh sách cho phép (allow-list) như khuyến nghị của OWASP [11].

3.4 Ví dụ 3: Tổ chức mã theo mẫu DAO, thao tác CRUD và giao dịch

Các ví dụ trước đặt câu SQL ngay trong phương thức main. Khi ứng dụng lớn dần, cách làm này khiến SQL rải rác khắp chương trình, khó kiểm thử và khó thay đổi. Ví dụ 3 áp dụng mẫu DAO (Mục 2.6.5) với bốn kiểu dữ liệu: lớp miền Student, interface StudentDao, lớp hiện thực JdbcStudentDao và lớp chạy thử DaoDemo. Quan hệ giữa chúng được thể hiện trong sơ đồ lớp UML (Unified Modeling Language) ở Hình 3.2.



Hình 3.2. Sơ đồ lớp UML của ví dụ DAO
Nguồn: Tác giả thiết kế

Student là lớp đối tượng miền với các trường private và các phương thức truy xuất (Mã nguồn B.1). Interface StudentDao (Mã nguồn 3.4) định nghĩa hợp đồng gồm ba thao tác đọc và ba thao tác ghi, bao phủ đủ bốn thao tác cơ bản tạo – đọc – sửa – xóa (Create, Read, Update, Delete – CRUD). Kiểu trả về Optional<Student> của findBySsn (dòng 7) biểu thị tường minh khả năng "không tìm thấy" thay vì trả về null. Các phương thức khai báo throws SQLException; đây là lựa chọn đơn giản phục vụ minh họa nhưng làm lộ chi tiết JDBC ra interface, và cách khắc phục được bàn ở Chương 4.

Mã nguồn 3.4. StudentDao.java – interface truy cập dữ liệu sinh viên

```
1  import java.sql.SQLException;
2  import java.util.List;
3  import java.util.Optional;
4
5  /** Hợp đồng truy cập dữ liệu; tầng trên chỉ phụ thuộc interface. */
6  public interface StudentDao {
7    Optional<Student> findBySsn(String ssn) throws SQLException;
8    List<Student> findByDept(String deptId) throws SQLException;
9    int insert(Student s) throws SQLException;
10   int updateDept(String ssn, String deptId) throws SQLException;
11   int delete(String ssn) throws SQLException;
12   /** Đăng ký nhiều môn trong MỘT giao dịch (tất cả hoặc không). */
13   void registerCourses(String ssn, List<String> courseIds)
14       throws SQLException;
15 }
```

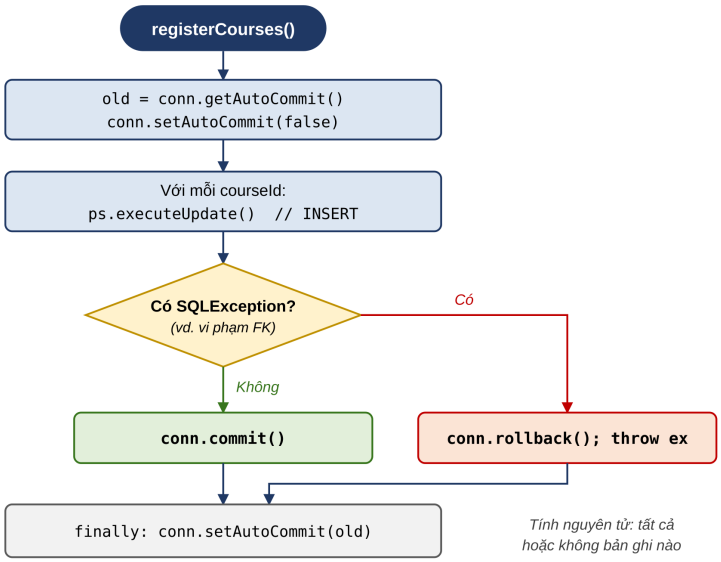
JdbcStudentDao (Mã nguồn B.2) hiện thực interface trên. Lớp nhận Connection qua constructor thay vì tự tạo, một dạng đơn giản của kỹ thuật tiêm phụ thuộc (dependency injection) cho phép nhiều DAO dùng chung một kết nối và một giao dịch. Mọi phương thức đều dùng PreparedStatement. Phương thức private static map chuyển một dòng

ResultSet thành đối tượng Student, xử lý cả trường hợp birthDate mang giá trị NULL như sinh viên 444111111 trong dữ liệu mẫu; việc chuyển đổi giữa java.sql.Date và java.time.LocalDate dùng Date.valueOf và toLocalDate. Mã nguồn 3.5 trích phương thức quan trọng nhất về mặt giao dịch.

Mã nguồn 3.5. Phương thức registerCourses trong JdbcStudentDao.java

```
 99  @Override
100  public void registerCourses(String ssn, List<String> courseIds)
101      throws SQLException {
102    String sql = "INSERT INTO Enrollment "
103            + "(ssn, courseId, dateRegistered) "
104            + "VALUES (?, ?, CURRENT_DATE)";
105    boolean oldAutoCommit = conn.getAutoCommit();
106    conn.setAutoCommit(false);            // bắt đầu giao dịch
107    try (PreparedStatement ps = conn.prepareStatement(sql)) {
108      for (String courseId : courseIds) {
109        ps.setString(1, ssn);
110        ps.setString(2, courseId);
111        ps.executeUpdate();
112      }
113      conn.commit();                      // tất cả thành công
114    } catch (SQLException ex) {
115      conn.rollback();                    // hủy mọi thay đổi
116      throw ex;                           // báo lỗi lên trên
117    } finally {
118      conn.setAutoCommit(oldAutoCommit);    // khôi phục trạng thái
119    }
120  }
```

Dòng 105–106 lưu trạng thái auto-commit hiện tại rồi tắt chế độ này; kể từ đây các lệnh INSERT thuộc cùng một giao dịch. Vòng lặp ở dòng 108–112 tái sử dụng một PreparedStatement cho nhiều môn học, đúng tình huống mà PreparedStatement phát huy hiệu quả. Nếu mọi lệnh thành công, dòng 113 xác nhận giao dịch. Nếu xảy ra SQLException, chẳng hạn mã môn 99999 không tồn tại nên vi phạm khóa ngoại, dòng 115 hủy toàn bộ giao dịch, kể cả những bản ghi đã chèn thành công trước đó, rồi dòng 116 ném lại ngoại lệ để tầng gọi biết thao tác thất bại. Khối finally (dòng 117–119) khôi phục chế độ auto-commit ban đầu. Toàn bộ luồng xử lý được tóm tắt trong Hình 3.3.



Hình 3.3. Lưu đồ xử lý giao dịch trong phương thức registerCourses
Nguồn: Tác giả thiết kế dựa trên [4] và [6]

Thứ tự rollback() trước, finally sau là bắt buộc chứ không chỉ là thói quen tốt. Như đã nêu ở Mục 2.3.3, nếu thiếu lời gọi rollback() thì lời gọi setAutoCommit(true) trong khối finally sẽ xác nhận phần giao dịch dở dang [6]: bản ghi đăng ký môn 11111 vẫn được lưu dù thao tác "đăng ký hai môn" đã thất bại. Phương thức delete áp dụng cùng khuôn mẫu, xóa bản ghi ở bảng con Enrollment trước rồi mới xóa ở bảng cha Student để không vi phạm khóa ngoại. Lớp DaoDemo (Mã nguồn B.3) chạy lần lượt các thao tác và cho kết quả dự kiến như sau:

Kết quả dự kiến:

```
Sinh vien khoa BIOL:
  444111120 Rick Carter (BIOL)
  444111113 Frank Jones (BIOL)
  444111110 Jacob Smith (BIOL)
  444111116 Josh Smith (BIOL)
  444111111 John Stevenson (BIOL)
insert -> 1 dong
update -> 1 dong
Tim lai: 444111199 An Nguyen (MATH)
Giao dich bi huy: Cannot add or update a child row: a foreign key
                  constraint fails (...)
Dang ky 2 mon thanh cong
delete -> 1 dong
```

Danh sách sinh viên khoa BIOL được sắp theo họ rồi đến tên, khớp với kết quả trên SQLite. Thông báo sau dòng "Giao dich bi huy" là thông báo lỗi vi phạm khóa ngoại của MySQL; nội dung chi tiết trong ngoặc phụ thuộc phiên bản. Việc lần đăng ký thứ hai (môn 11111 và 11112) thành công là bằng chứng gián tiếp cho thấy bản ghi môn 11111 của lần trước đã bị hủy; nếu không, lần thứ hai sẽ vi phạm khóa chính (ssn,

courseId). Hành vi hủy giao dịch này đã được kiểm chứng trên SQLite: sau lỗi khóa ngoại và lệnh rollback, sinh viên mới không còn bản ghi đăng ký nào.

Về mặt thiết kế, ngoài dòng khởi tạo new JdbcStudentDao(conn), lớp DaoDemo chỉ phụ thuộc vào interface StudentDao. Có thể thay JdbcStudentDao bằng một hiện thực khác, ví dụ một DAO giả lập lưu dữ liệu trong bộ nhớ để kiểm thử, mà không phải sửa phần còn lại của chương trình. Đây là đa hình được vận dụng ở cấp độ thiết kế, cùng tư tưởng với thiết kế của chính JDBC.

3.5 Ví dụ 4: Gọi hàm lưu trữ bằng CallableStatement

Giáo trình minh họa CallableStatement bằng hàm studentFound(first, last) trả về số sinh viên có tên và họ tương ứng [1, pp. 1200–1201]. Mã nguồn 3.6 là phiên bản của hàm dành cho MySQL 8.

Mã nguồn 3.6. Hàm lưu trữ studentFound (MySQL 8)

```
 1  DROP FUNCTION IF EXISTS studentFound;
 2  DELIMITER //
 3  CREATE FUNCTION studentFound(first VARCHAR(20), last VARCHAR(20))
 4    RETURNS INT
 5    READS SQL DATA
 6  BEGIN
 7    DECLARE result INT;
 8    SELECT COUNT(*) INTO result
 9    FROM Student
10    WHERE Student.firstName = first AND Student.lastName = last;
11    RETURN result;
12  END //
13  DELIMITER ;
```

Khác với phiên bản trong giáo trình, hàm được khai báo thêm READS SQL DATA (dòng 5). Khi ghi nhật ký nhị phân (binary log) được bật, MySQL từ chối câu lệnh CREATE FUNCTION không khai báo ít nhất một trong các đặc tính DETERMINISTIC, NO SQL hoặc READS SQL DATA, và báo lỗi 1418 [9]. Binary log được bật mặc định trên MySQL 8, nên script của giáo trình có thể gặp lỗi này. Hàm studentFound chỉ đọc dữ liệu, vì vậy READS SQL DATA là khai báo đúng với ngữ nghĩa của nó. Chương trình Java gọi hàm được trình bày ở Mã nguồn 3.7.

Mã nguồn 3.7. TestCallableStatement.java – gọi hàm lưu trữ

```
 1  import java.sql.CallableStatement;
 2  import java.sql.Connection;
 3  import java.sql.SQLException;
 4  import java.sql.Types;
 5
 6  public class TestCallableStatement {
 7    public static void main(String[] args) {
 8      String call = "{? = call studentFound(?, ?)}";  // cú pháp escape
 9      String[][] names = { {"Jacob", "Smith"}, {"John", "Smith"} };
10      try (Connection conn = DbConfig.getConnection();
11           CallableStatement cs = conn.prepareCall(call)) {
12        cs.registerOutParameter(1, Types.INTEGER);  // giá trị trả về
13        for (String[] n : names) {
14          cs.setString(2, n[0]);
15          cs.setString(3, n[1]);
16          cs.execute();
17          String verb = cs.getInt(1) >= 1 ? " is" : " is not";
18          System.out.println(n[0] + " " + n[1] + verb
19              + " in the database");
20        }
21      } catch (SQLException ex) {
22        System.err.println("Loi CSDL: " + ex.getMessage());
23      }
24    }
25  }
```

Dòng 8 viết câu gọi theo cú pháp escape {? = call studentFound(?, ?)}: dấu ? đầu tiên đại diện cho giá trị trả về nên mang chỉ số 1, hai đối số của hàm mang chỉ số 2 và 3. Dòng 12 đăng ký kiểu INTEGER cho tham số OUT số 1; việc đăng ký chỉ cần thực hiện một lần bên ngoài vòng lặp vì cùng một đối tượng CallableStatement được tái sử dụng. Dòng 14–16 gán hai tham số IN và thực thi; dòng 17 đọc giá trị trả về bằng getInt(1).

Kết quả dự kiến:

```
Jacob Smith is in the database
John Smith is not in the database
```

Kết quả được suy ra từ dữ liệu mẫu: CSDL có đúng một sinh viên Jacob Smith (444111110) và không có John Smith (chỉ có John Stevenson); câu truy vấn COUNT(*) tương đương trên SQLite cho kết quả lần lượt là 1 và 0. Riêng việc gọi hàm lưu trữ là đặc thù của MySQL nên phần này chưa được kiểm chứng trực tiếp. Về đánh giá, việc dùng hàm và thủ tục lưu trữ giúp đưa logic về phía DBMS, giảm số lượt trao đổi qua mạng và tập trung quy tắc nghiệp vụ tại một nơi; đổi lại, ứng dụng phụ thuộc vào ngôn ngữ thủ tục riêng của từng DBMS, đi ngược với tính độc lập mà JDBC hướng tới. Lựa chọn phù hợp tùy thuộc vào ngữ cảnh của từng hệ thống.

3.6 Ví dụ 5: Khai thác siêu dữ liệu

Ví dụ cuối cùng kết hợp nội dung của các Listing 32.5–32.7 trong giáo trình [1, pp. 1202–1205] thành một chương trình duy nhất, đồng thời tách phần in bảng thành một phương thức tổng quát có thể tái sử dụng.

Mã nguồn 3.8. MetadataDemo.java – đọc siêu dữ liệu CSDL và kết quả truy vấn

```
 1  import java.sql.Connection;
 2  import java.sql.DatabaseMetaData;
 3  import java.sql.ResultSet;
 4  import java.sql.ResultSetMetaData;
 5  import java.sql.SQLException;
 6  import java.sql.Statement;
 7
 8  public class MetadataDemo {
 9    public static void main(String[] args) {
10      try (Connection conn = DbConfig.getConnection()) {
11        DatabaseMetaData db = conn.getMetaData();
12        System.out.println("DBMS   : " + db.getDatabaseProductName()
13            + " " + db.getDatabaseProductVersion());
14        System.out.println("Driver : " + db.getDriverName()
15            + " " + db.getDriverVersion());
16        System.out.println("JDBC   : " + db.getJDBCMajorVersion()
17            + "." + db.getJDBCMinorVersion());
18
19        System.out.print("Bang   :");
20        try (ResultSet t = db.getTables(conn.getCatalog(), null, "%",
21                                        new String[] {"TABLE"})) {
22          while (t.next()) {
23            System.out.print(" " + t.getString("TABLE  NAME"));
24          }
25        }
26        System.out.println();
27        printTable(conn,
28            "SELECT * FROM Course WHERE subjectId = 'CSCI'");
29      } catch (SQLException ex) {
30        System.err.println("Loi CSDL: " + ex.getMessage());
31      }
32    }
33
34    /** In một truy vấn bất kỳ mà không cần biết trước các cột. */
35    static void printTable(Connection conn, String sql)
36        throws SQLException {
37      try (Statement st = conn.createStatement();
38           ResultSet rs = st.executeQuery(sql)) {
39        ResultSetMetaData md = rs.getMetaData();
40        int n = md.getColumnCount();
41        for (int i = 1; i <= n; i++) {       // chỉ số cột bắt đầu từ 1
42          System.out.print(md.getColumnLabel(i) + (i < n ? "\t" : "\n"));
43        }
44        while (rs.next()) {
45          for (int i = 1; i <= n; i++) {
46            System.out.print(rs.getObject(i) + (i < n ? "\t" : "\n"));
47          }
48        }
49      }
50    }
51  }
```

Dòng 11 lấy đối tượng DatabaseMetaData từ Connection; dòng 12–17 đọc thông tin về sản phẩm, driver và phiên bản JDBC. Dòng 20–21 gọi getTables với catalog hiện

hành, mẫu tên "%" (mọi bảng) và loại "TABLE" (loại trừ view và bảng hệ thống). Kết quả là một ResultSet thông thường, được duyệt như mọi ResultSet khác và đọc cột TABLE_NAME (dòng 23). Phương thức printTable (dòng 35–50) là ví dụ về mã tổng quát: nó không biết câu SQL truy vấn bảng nào, nhưng nhờ ResultSetMetaData biết số cột (dòng 40) và nhãn cột (dòng 42). Lời gọi rs.getObject(i) (dòng 46) đọc mọi kiểu dữ liệu dưới dạng Object, và việc chuyển thành chuỗi khi in dựa vào phương thức toString() được ghi đè ở từng lớp, một ứng dụng nữa của đa hình.

Kết quả dự kiến:

```
DBMS   : MySQL <phiên bản máy chủ, ví dụ 8.0.x>
Driver : MySQL Connector/J <phiên bản driver>
JDBC   : <phiên bản JDBC mà driver hỗ trợ>
Bang   : course enrollment student
courseId  subjectId  courseNumber  title                numOfCredits
11111     CSCI       1301          Introduction to Java I  4
11112     CSCI       1302          Introduction to Java II 3
11113     CSCI       3720          Database Systems        3
11114     CSCI       4750          Rapid Java Application  3
```

Ba dòng đầu phụ thuộc phiên bản phần mềm được cài đặt nên được ghi dưới dạng <...>. Tên bảng có thể hiển thị bằng chữ thường hoặc giữ nguyên cách viết ban đầu tùy hệ điều hành và biến cấu hình lower_case_table_names của MySQL. Chương trình in các cột cách nhau bằng ký tự tab; trong khối kết quả trên, các cột được căn thẳng để dễ đọc. Bốn môn học CSCI khớp với kết quả truy vấn trên SQLite.

3.7 Các lỗi thường gặp và cách xử lý

Qua quá trình xây dựng các ví dụ và đối chiếu tài liệu, Bảng 3.2 tổng hợp những lỗi thường gặp khi lập trình JDBC, nguyên nhân và cách xử lý tương ứng.

Bảng 3.2. Các lỗi thường gặp khi lập trình JDBC

Lỗi / triệu chứng	Nguyên nhân	Cách xử lý
No suitable driver found for jdbc:mysql://...	File JAR của driver không có trong class path khi chạy, hoặc URL sai tiền tố	Thêm JAR vào class path bằng -cp; kiểm tra URL (Phụ lục C)
ClassNotFoundException khi gọi Class.forName, hoặc cảnh báo lớp driver đã lỗi thời	Thiếu JAR, hoặc dùng tên lớp com.mysql.jdbc.Driver của Connector/J 5.1	Dùng com.mysql.cj.jdbc.Driver [8] hoặc bỏ Class.forName
Access denied for user ...	Sai tài khoản, mật khẩu hoặc tài khoản chưa được cấp quyền trên CSDL	Kiểm tra thông tin; cấp quyền bằng GRANT (Phụ lục C)

Lỗi / triệu chứng	Nguyên nhân	Cách xử lý
SQLException khi gọi getXxx ngay sau executeQuery	Con trỏ còn ở trước dòng đầu tiên do quên gọi next()	Luôn gọi rs.next() và kiểm tra kết quả trước khi đọc
Lỗi chỉ số cột hoặc chỉ số tham số không hợp lệ	Đánh chỉ số từ 0 như mảng Java	Chỉ số cột và tham số ? bắt đầu từ 1
SQLException khi gọi ps.executeQuery(sql)	Gọi nhầm phương thức có đối số kế thừa từ Statement trên PreparedStatement	Gọi executeQuery() hoặc executeUpdate() không đối số
Lỗi "Too many connections" sau một thời gian chạy	Không đóng Connection, Statement trên nhánh ngoại lệ (rò rỉ tài nguyên)	Dùng try-with-resources cho mọi tài nguyên JDBC
Lỗi 1418 khi tạo hàm lưu trữ	Binary log được bật; hàm không khai báo đặc tính	Thêm READS SQL DATA hoặc DETERMINISTIC [9]
Dữ liệu bị ghi dở dang khi lỗi giữa chừng	Để chế độ auto-commit hoặc thiếu rollback()	Dùng setAutoCommit(false), commit(), rollback() (Mục 3.4)
Lỗi hiển thị tiếng Việt	Bảng mã của cửa sổ dòng lệnh hoặc của CSDL không phải UTF-8	Dùng bảng mã utf8mb4 cho CSDL; đặt bảng mã UTF-8 cho cửa sổ dòng lệnh

Nguồn: Tác giả tổng hợp; lỗi ở dòng thứ nhất được quan sát trực tiếp khi chạy chương trình (Mục 3.2)

3.8 So sánh các cách tiếp cận

Mục này so sánh hai cặp lựa chọn xuất hiện xuyên suốt chương: cách đóng tài nguyên (Bảng 3.3) và loại câu lệnh (Bảng 3.4). Mục đích không phải là chọn ra một phương án "tốt nhất" trong mọi trường hợp, mà là làm rõ ưu thế của từng phương án trong những ngữ cảnh khác nhau.

Bảng 3.3. So sánh đóng tài nguyên thủ công và try-with-resources

Tiêu chí	try–finally, đóng thủ công (cách của [1])	try-with-resources
Độ dài mã	Dài; cần kiểm tra null và lồng try vì close() cũng ném SQLException	Ngắn; khai báo tài nguyên trong cặp ngoặc của try
Thứ tự đóng	Do lập trình viên tự bảo đảm	Tự động, ngược với thứ tự khai báo
Ngoại lệ khi đóng	Có thể che mất ngoại lệ gốc	Được lưu dưới dạng suppressed, ngoại lệ gốc được giữ nguyên [5]
Yêu cầu	Mọi phiên bản Java	Java 7 trở lên; tài nguyên phải hiện thực AutoCloseable
Ngữ cảnh phù hợp	Mã cũ; tài nguyên cần giữ mở vượt ra ngoài một khối lệnh	Phần lớn mã mới, khi tài nguyên được dùng trong phạm vi một khối

Nguồn: Tác giả tổng hợp từ [1], [5]

Bảng 3.4. So sánh ba loại câu lệnh của JDBC

Tiêu chí	Statement	PreparedStatement	CallableStatement
Cách tạo	createStatement()	prepareStatement(sql)	prepareCall(sql)
Câu SQL	Truyền khi thực thi	Gắn khi tạo, có tham số ?	Cú pháp escape {call ...}
Loại tham số	Không có	IN	IN, OUT, IN OUT
Chống SQL injection	Không, nếu ghép chuỗi đầu vào	Có, đối với giá trị tham số	Có với tham số; mã bên trong thủ tục vẫn phải viết an toàn [11]
Biên dịch trước	Không	Có (tùy driver)	Có (tùy driver)
Ngữ cảnh phù hợp	Câu DDL hoặc câu tĩnh không chứa đầu vào từ bên ngoài	Phần lớn truy vấn và cập nhật có tham số	Logic nghiệp vụ được đặt trong DBMS

Nguồn: Tác giả tổng hợp từ [1], [6], [11]

Có thể thấy, Statement vẫn hợp lý cho các câu DDL cố định như trong script khởi tạo, còn CallableStatement đánh đổi tính khả chuyển để tập trung logic tại DBMS. Tuy vậy, đối với câu lệnh chứa dữ liệu do người dùng nhập, PreparedStatement là lựa chọn mặc định an toàn hơn theo khuyến nghị của OWASP [11]. Tương tự, try-with-resources phù hợp với đa số tình huống, nhưng khi một tài nguyên cần tồn tại lâu hơn một khối lệnh, chẳng hạn kết nối được truyền vào DAO ở Ví dụ 3, việc đóng phải do đoạn mã sở hữu tài nguyên đó đảm nhận.

CHƯƠNG 4. ĐÁNH GIÁ VÀ ỨNG DỤNG

Dựa trên phân tích lý thuyết và các ví dụ ở hai chương trước, chương này đánh giá ưu điểm, hạn chế của việc lập trình CSDL trực tiếp bằng JDBC, các hướng ứng dụng thực tế và những lưu ý khi áp dụng vào đồ án học phần.

4.1 Ưu điểm

Ưu điểm nổi bật nhất của JDBC là tính độc lập với hệ quản trị ở mức API: ứng dụng chỉ làm việc với các interface trong java.sql, còn việc giao tiếp với từng DBMS do driver đảm nhận (Mục 2.2). JDBC là một phần của nền tảng Java SE nên không cần thêm framework nào, chỉ cần driver của DBMS. Lập trình viên kiểm soát hoàn toàn câu SQL được gửi đi, thuận lợi cho việc tối ưu truy vấn và gỡ lỗi, vì những gì được thực thi chính là những gì được viết trong mã. Ngoài ra, JDBC là tầng nền mà các công nghệ ở mức cao hơn sử dụng bên dưới, nên hiểu JDBC giúp chẩn đoán đúng các vấn đề như rò rỉ kết nối hay giao dịch sai phạm vi khi làm việc với những công nghệ đó.

4.2 Hạn chế

Hạn chế đầu tiên là lượng mã lặp lại (boilerplate) đáng kể: mở và đóng tài nguyên, gán tham số, ánh xạ thủ công từng dòng ResultSet thành đối tượng như phương thức map trong JdbcStudentDao. Hạn chế thứ hai là sự lệch pha giữa mô hình đối tượng và mô hình quan hệ (object-relational impedance mismatch): các khái niệm như kế thừa hay liên kết giữa đối tượng không có tương ứng trực tiếp trong bảng, và lập trình viên phải tự xử lý việc chuyển đổi. Thứ ba, câu SQL nằm trong chuỗi ký tự nên lỗi cú pháp hay sai tên cột chỉ lộ ra khi chạy, trình biên dịch Java không thể phát hiện. Thứ tư, tính khả chuyển chỉ dừng ở mức API; các câu SQL dùng hàm riêng hoặc thủ tục lưu trữ vẫn gắn với một DBMS cụ thể (Mục 3.5). Cuối cùng, vì SQLException là checked exception, khi lan truyền qua interface DAO nó làm lộ chi tiết hạ tầng ra các tầng phía trên, như đã thấy ở Mã nguồn 3.4.

4.3 Ứng dụng thực tế

JDBC được dùng trực tiếp trong các ứng dụng quản lý quy mô nhỏ và vừa như quản lý sinh viên, thư viện hay bán hàng, dưới dạng ứng dụng máy tính để bàn hoặc phần xử lý phía máy chủ của ứng dụng web. Các công cụ quản trị và di chuyển dữ liệu

khai thác siêu dữ liệu để làm việc với những CSDL mà cấu trúc chưa được biết trước, tương tự phương thức printTable ở Ví dụ 5.

Trong các hệ thống lớn hơn, JDBC thường được sử dụng gián tiếp qua nhiều lớp hỗ trợ. Interface DataSource, cách tạo kết nối được Oracle khuyến nghị [3], kết hợp với connection pool cho phép tái sử dụng kết nối thay vì mở kết nối mới cho mỗi yêu cầu, vốn là thao tác tương đối tốn kém. Các thư viện như Spring JdbcTemplate giảm lượng mã lặp lại, còn các ORM như Hibernate (hiện thực đặc tả JPA) tự động hóa việc ánh xạ đối tượng – bảng. Những công nghệ này không thay thế JDBC mà xây dựng trên JDBC; việc lựa chọn mức trừu tượng nào phụ thuộc vào quy mô dự án, yêu cầu hiệu năng và kinh nghiệm của nhóm phát triển.

4.4 Lưu ý khi áp dụng vào đồ án học phần

Đối với đồ án nhóm của học phần, các ví dụ trong báo cáo gợi ý một số nguyên tắc có thể áp dụng ngay:

1. Tổ chức mã theo tầng: lớp miền (model), lớp truy cập dữ liệu (DAO), lớp xử lý nghiệp vụ và lớp giao diện; giao diện không chứa câu SQL.
2. Mọi câu lệnh chứa dữ liệu từ người dùng đều dùng PreparedStatement; với tên cột hay chiều sắp xếp, kiểm tra theo danh sách cho phép.
3. Dùng try-with-resources cho mọi tài nguyên JDBC và gom các thao tác nhiều bước vào một giao dịch có rollback() trong nhánh lỗi.
4. Đặt thông tin kết nối ngoài mã nguồn (thuộc tính hệ thống, tệp cấu hình), không đưa mật khẩu lên kho mã dùng chung; tạo tài khoản CSDL riêng với quyền tối thiểu.
5. Lưu script tạo CSDL và dữ liệu mẫu trong dự án để mọi thành viên dựng lại được cùng một môi trường.

Để khắc phục hạn chế về việc lộ SQLException qua interface, DAO có thể bọc ngoại lệ này trong một ngoại lệ riêng của ứng dụng thuộc loại unchecked; khi đó các tầng phía trên không còn phụ thuộc vào gói java.sql.

4.5 Liên hệ với nội dung học phần

Bảng 4.1 tổng kết mối liên hệ giữa các chương của bài giảng học phần [2] và những gì đã được phân tích trong báo cáo, cho thấy JDBC là nơi hội tụ của nhiều kiến thức hướng đối tượng.

Bảng 4.1. Liên hệ giữa nội dung bài giảng và chủ đề JDBC

Nội dung bài giảng [2]	Biểu hiện trong JDBC và trong báo cáo
Chương 4 – Đối tượng và lớp: đóng gói, class path, gợi ý thiết kế lớp	Lớp Student với trường private; DAO tách trách nhiệm; JAR của driver phải có trong class path; DbConfig là lớp tiện ích
Chương 5 – Kế thừa: đa hình, liên kết động, reflection	Lời gọi trên biến kiểu interface được liên kết động tới lớp của driver; Class.forName nạp driver theo tên
Chương 6 – Interface và biểu thức lambda	java.sql gần như toàn interface; PreparedStatement extends Statement; interface StudentDao; lambda trong forEach của DaoDemo
Chương 7 – Ngoại lệ	SQLException là checked exception; try-with-resources; rollback() trong khối catch
Chương 9 – Collections	List<Student> làm kiểu trả về; tách interface khỏi hiện thực tương tự List/ArrayList và StudentDao/JdbcStudentDao

Nguồn: Tác giả tổng hợp từ [2] và nội dung Chương 2, 3

CHƯƠNG 5. KẾT LUẬN

5.1 Kết quả đạt được

Báo cáo đã trình bày một cách hệ thống chủ đề lập trình CSDL với Java. Về lý thuyết, báo cáo làm rõ kiến trúc hai tầng của JDBC, vai trò của bốn interface cốt lõi, quy trình truy cập CSDL, cơ chế giao dịch, ba loại câu lệnh và hai loại siêu dữ liệu; đồng thời cập nhật những điểm đã thay đổi so với giáo trình như tên lớp driver của Connector/J 8, cơ chế tự động phát hiện driver, việc loại bỏ cầu nối JDBC-ODBC và yêu cầu khai báo hàm lưu trữ trong MySQL 8. Về góc nhìn hướng đối tượng, báo cáo chỉ ra JDBC là minh họa thực tế của trừu tượng hóa qua interface, đa hình, kế thừa interface, reflection và xử lý ngoại lệ có kiểm tra.

Về thực hành, báo cáo xây dựng năm ví dụ Java biên dịch được, trong đó Ví dụ 2 chứng minh cụ thể cơ chế của tấn công SQL injection và cách PreparedStatement ngăn chặn nó, còn Ví dụ 3 vận dụng mẫu DAO và chỉ ra một chi tiết dễ bị bỏ qua của đặc tả: thiếu rollback() có thể dẫn đến việc khối finally vô tình xác nhận một giao dịch dở dang. Các bảng lỗi thường gặp và bảng so sánh giúp chuyển kiến thức thành những nguyên tắc có thể áp dụng trong đồ án.

5.2 Hạn chế của báo cáo

Báo cáo còn một số hạn chế. Các chương trình chưa được chạy trên máy chủ MySQL thật; kết quả được trình bày dưới dạng dự kiến và chỉ được đối chiếu một phần bằng SQLite, riêng phần hàm lưu trữ và các giá trị siêu dữ liệu phụ thuộc phiên bản chưa được kiểm chứng. Báo cáo chưa đo đạc hiệu năng nên không đưa ra kết luận định lượng về lợi ích hiệu năng của PreparedStatement. Một số chủ đề như connection pool, DataSource, xử lý theo lô và ResultSet có thể cuộn hay cập nhật chưa được phân tích.

5.3 Hướng phát triển

Bước tiếp theo trước hết là chạy các ví dụ trên MySQL 8 theo Phụ lục C để thay kết quả dự kiến bằng kết quả thực tế. Tiếp đó, có thể viết kiểm thử tự động cho các DAO bằng JUnit kết hợp một CSDL nhúng, bổ sung connection pool và đo so sánh hiệu năng giữa Statement và PreparedStatement trong các kịch bản lặp lại. Ở mức ứng dụng, các ví dụ có thể được mở rộng thành một ứng dụng quản lý sinh viên có giao diện đồ họa cho

đồ án học phần, và sau đó được so sánh với một phiên bản dùng JPA để thấy rõ sự đánh đổi giữa kiểm soát chi tiết và mức độ tự động hóa.

TÀI LIỆU THAM KHẢO

[1]Y. D. Liang, "Java database programming," in *Introduction to Java Programming, Comprehensive Version*, 10th ed. Boston, MA, USA: Pearson, 2015, ch. 32, pp. 1173–1212.

[2]Nguyễn Thị Tuyết Hải (biên soạn), *Bài giảng Lập trình hướng đối tượng*, các chương 4, 5, 6, 7 và 9 [Slide bài giảng học phần], n.d.

[3]Oracle, "Lesson: JDBC Basics," *The Java Tutorials*. [Online]. Available: https://docs.oracle.com/javase/tutorial/jdbc/basics/index.html (accessed Oct. 13, 2026).

[4]Oracle, "Using Transactions," *The Java Tutorials*. [Online]. Available: https://docs.oracle.com/javase/tutorial/jdbc/basics/transactions.html (accessed Oct. 13, 2026).

[5]Oracle, "The try-with-resources Statement," *The Java Tutorials*. [Online]. Available: https://docs.oracle.com/javase/tutorial/essential/exceptions/tryResourceClose.html (accessed Oct. 13, 2026).

[6]Oracle, "Package java.sql," *Java SE 21 & JDK 21 API Specification*. [Online]. Available: https://docs.oracle.com/en/java/javase/21/docs/api/java.sql/java/sql/package-summary.html (accessed Oct. 13, 2026).

[7]L. Andersen, "JSR 221: JDBC 4.0 API Specification," Java Community Process. [Online]. Available: https://jcp.org/en/jsr/detail?id=221 (accessed Oct. 13, 2026).

[8]Oracle, "Connecting to MySQL Using the JDBC DriverManager Interface," *MySQL Connector/J Developer Guide*. [Online]. Available: https://dev.mysql.com/doc/connector-j/en/connector-j-usagenotes-connect-drivermanager.html (accessed Oct. 13, 2026).

[9]Oracle, "Stored Program Binary Logging," *MySQL 8.0 Reference Manual*. [Online]. Available: https://dev.mysql.com/doc/refman/8.0/en/stored-programs-logging.html (accessed Oct. 13, 2026).

[10]Oracle, "Compatibility Guide for JDK 8." [Online]. Available: https://www.oracle.com/java/technologies/javase/8-compatibility-guide.html (accessed Oct. 13, 2026).

[11]OWASP Foundation, “SQL Injection Prevention Cheat Sheet,” *OWASP Cheat Sheet Series*. [Online]. Available: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html (accessed Oct. 13, 2026).

PHỤ LỤC A. SCRIPT TẠO CƠ SỞ DỮ LIỆU MẪU

Script dưới đây tạo CSDL javabook trên MySQL 8, gồm ba bảng Course, Student, Enrollment với dữ liệu theo [1, pp. 1175–1177] và hàm lưu trữ studentFound. Script xóa và tạo lại các bảng, vì vậy có thể chạy lại nhiều lần để khôi phục dữ liệu ban đầu.

Mã nguồn A.1. javabook.sql – tạo CSDL, bảng, dữ liệu mẫu và hàm lưu trữ

```
-- Tạo CSDL và dữ liệu mẫu cho các ví dụ (MySQL 8)
CREATE DATABASE IF NOT EXISTS javabook;
USE javabook;

DROP TABLE IF EXISTS Enrollment;
DROP TABLE IF EXISTS Student;
DROP TABLE IF EXISTS Course;

CREATE TABLE Course (
  courseId     CHAR(5),
  subjectId    CHAR(4) NOT NULL,
  courseNumber INTEGER,
  title        VARCHAR(50) NOT NULL,
  numOfCredits INTEGER,
  PRIMARY KEY (courseId)
);

CREATE TABLE Student (
  ssn       CHAR(9),
  firstName VARCHAR(25),
  mi        CHAR(1),
  lastName  VARCHAR(25),
  birthDate DATE,
  street    VARCHAR(25),
  phone     CHAR(11),
  zipCode   CHAR(5),
  deptId    CHAR(4),
  PRIMARY KEY (ssn)
);

CREATE TABLE Enrollment (
  ssn            CHAR(9),
  courseId       CHAR(5),
  dateRegistered DATE,
  grade          CHAR(1),
  PRIMARY KEY (ssn, courseId),
  FOREIGN KEY (ssn) REFERENCES Student(ssn),
  FOREIGN KEY (courseId) REFERENCES Course(courseId)
);

INSERT INTO Course VALUES
 ('11111','CSCI',1301,'Introduction to Java I',4),
 ('11112','CSCI',1302,'Introduction to Java II',3),
 ('11113','CSCI',3720,'Database Systems',3),
 ('11114','CSCI',4750,'Rapid Java Application',3),
 ('11115','MATH',2750,'Calculus I',5),
 ('11116','MATH',3750,'Calculus II',5),
 ('11117','EDUC',1111,'Reading',3),
 ('11118','ITEC',1344,'Database Administration',3);

INSERT INTO Student VALUES
 ('444111110','Jacob','R','Smith','1985-04-09',
  '99 Kingston Street','9129219434','31435','BIOL'),
 ('444111111','John','K','Stevenson',NULL,
  '100 Main Street','9129219434','31411','BIOL'),
 ('444111112','George','K','Smith','1974-10-10',
```

```
  '1200 Abercorn St.','9129213454','31419','CS'),
('444111113','Frank','E','Jones','1970-09-09',
  '100 Main Street','9125919434','31411','BIOL'),
('444111114','Jean','K','Smith','1970-02-09',
  '100 Main Street','9129219434','31411','CHEM'),
('444111115','Josh','R','Woo','1970-02-09',
  '555 Franklin St.','7075989434','31411','CHEM'),
('444111116','Josh','R','Smith','1973-02-09',
  '100 Main Street','9129219434','31411','BIOL'),
('444111117','Joy','P','Kennedy','1974-03-19',
  '103 Bay Street','9129229434','31412','CS'),
('444111118','Toni','R','Peterson','1964-04-29',
  '103 Bay Street','9129229434','31412','MATH'),
('444111119','Patrick','R','Stoneman','1969-04-29',
  '101 Washington St.','9129229434','31435','MATH'),
('444111120','Rick','R','Carter','1986-04-09',
  '19 West Ford St.','9125919434','31411','BIOL');

INSERT INTO Enrollment VALUES
('444111110','11111','2004-03-19','A'),
('444111110','11112','2004-03-19','B'),
('444111110','11113','2004-03-19','C'),
('444111111','11111','2004-03-19','D'),
('444111111','11112','2004-03-19','F'),
('444111111','11113','2004-03-19','A'),
('444111112','11114','2004-03-19','B'),
('444111112','11115','2004-03-19','C'),
('444111112','11116','2004-03-19','D'),
('444111113','11111','2004-03-19','A'),
('444111113','11113','2004-03-19','A'),
('444111114','11115','2004-03-19','B'),
('444111115','11115','2004-03-19','F'),
('444111115','11116','2004-03-19','F'),
('444111116','11111','2004-03-19','D'),
('444111117','11111','2004-03-19','D'),
('444111118','11111','2004-03-19','A'),
('444111118','11112','2004-03-19','D'),
('444111118','11113','2004-03-19','B');

-- Hàm lưu trữ cho Ví dụ 4. READS SQL DATA là bắt buộc khi MySQL 8
-- bật binary log (mặc định).
DROP FUNCTION IF EXISTS studentFound;
DELIMITER //
CREATE FUNCTION studentFound(first VARCHAR(20), last VARCHAR(20))
  RETURNS INT
  READS SQL DATA
BEGIN
  DECLARE result INT;
  SELECT COUNT(*) INTO result
  FROM Student
  WHERE Student.firstName = first AND Student.lastName = last;
  RETURN result;
END //
DELIMITER ;
```

PHỤ LỤC B. MÃ NGUỒN ĐẦY ĐỦ CỦA VÍ DỤ DAO

Phụ lục này trình bày đầy đủ ba tệp của Ví dụ 3 (Mục 3.4) chưa được in trong chương chính. Cùng với DbConfig.java (Mã nguồn 3.1) và StudentDao.java (Mã nguồn 3.4), chúng tạo thành một chương trình hoàn chỉnh.

Mã nguồn B.1. Student.java – lớp đối tượng miền

```
import java.time.LocalDate;

/** Đối tượng miền (domain object) ứng với một dòng bảng Student. */
public class Student {
  private final String ssn;
  private final String firstName;
  private final String lastName;
  private final LocalDate birthDate;
  private String deptId;

  public Student(String ssn, String firstName, String lastName,
                 LocalDate birthDate, String deptId) {
    this.ssn = ssn;
    this.firstName = firstName;
    this.lastName = lastName;
    this.birthDate = birthDate;
    this.deptId = deptId;
  }

  public String getSsn() { return ssn; }
  public String getFirstName() { return firstName; }
  public String getLastName() { return lastName; }
  public LocalDate getBirthDate() { return birthDate; }
  public String getDeptId() { return deptId; }
  public void setDeptId(String deptId) { this.deptId = deptId; }

  @Override
  public String toString() {
    return ssn + " " + firstName + " " + lastName
        + " (" + deptId + ")";
  }
}
```

Mã nguồn B.2. JdbcStudentDao.java – hiện thực StudentDao bằng JDBC

```
import java.sql.Connection;
import java.sql.Date;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.List;
import java.util.Optional;

/** Hiện thực StudentDao bằng JDBC, chỉ dùng PreparedStatement. */
public class JdbcStudentDao implements StudentDao {
  private static final String COLS =
      "SELECT ssn, firstName, lastName, birthDate, deptId "
    + "FROM Student ";
  private final Connection conn;

  public JdbcStudentDao(Connection conn) { this.conn = conn; }

```

```
20  @Override
21  public Optional<Student> findBySsn(String ssn)
22      throws SQLException {
23    try (PreparedStatement ps =
24            conn.prepareStatement(COLS + "WHERE ssn = ?")) {
25      ps.setString(1, ssn);
26      try (ResultSet rs = ps.executeQuery()) {
27        return rs.next() ? Optional.of(map(rs)) : Optional.empty();
28      }
29    }
30  }
31
32  @Override
33  public List<Student> findByDept(String deptId)
34      throws SQLException {
35    String sql = COLS + "WHERE deptId = ? "
36              + "ORDER BY lastName, firstName";
37    List<Student> list = new ArrayList<>();
38    try (PreparedStatement ps = conn.prepareStatement(sql)) {
39      ps.setString(1, deptId);
40      try (ResultSet rs = ps.executeQuery()) {
41        while (rs.next()) {
42          list.add(map(rs));
43        }
44      }
45    }
46    return list;
47  }
48
49  @Override
50  public int insert(Student s) throws SQLException {
51    String sql = "INSERT INTO Student (ssn, firstName, lastName, "
52              + "birthDate, deptId) VALUES (?, ?, ?, ?, ?)";
53    try (PreparedStatement ps = conn.prepareStatement(sql)) {
54      ps.setString(1, s.getSsn());
55      ps.setString(2, s.getFirstName());
56      ps.setString(3, s.getLastName());
57      LocalDate d = s.getBirthDate();
58      ps.setDate(4, d == null ? null : Date.valueOf(d));
59      ps.setString(5, s.getDeptId());
60      return ps.executeUpdate();     // số dòng bị ảnh hưởng
61    }
62  }
63
64  @Override
65  public int updateDept(String ssn, String deptId)
66      throws SQLException {
67    String sql = "UPDATE Student SET deptId = ? WHERE ssn = ?";
68    try (PreparedStatement ps = conn.prepareStatement(sql)) {
69      ps.setString(1, deptId);
70      ps.setString(2, ssn);
71      return ps.executeUpdate();
72    }
73  }
74
75  @Override
76  public int delete(String ssn) throws SQLException {
77    // Xóa bảng con (Enrollment) trước, bảng cha (Student) sau,
78    // trong cùng một giao dịch.
79    boolean oldAutoCommit = conn.getAutoCommit();
80    conn.setAutoCommit(false);
81    try (PreparedStatement e = conn.prepareStatement(
82            "DELETE FROM Enrollment WHERE ssn = ?");
83         PreparedStatement s = conn.prepareStatement(
84            "DELETE FROM Student WHERE ssn = ?")) {
85      e.setString(1, ssn);
86      e.executeUpdate();
87      s.setString(1, ssn);
88      int n = s.executeUpdate();
```

```
89        conn.commit();
90        return n;
91      } catch (SQLException ex) {
92        conn.rollback();
93        throw ex;
94      } finally {
95        conn.setAutoCommit(oldAutoCommit);
96      }
97    }
98
99    @Override
100   public void registerCourses(String ssn, List<String> courseIds)
101       throws SQLException {
102     String sql = "INSERT INTO Enrollment "
103             + "(ssn, courseId, dateRegistered) "
104             + "VALUES (?, ?, CURRENT_DATE)";
105     boolean oldAutoCommit = conn.getAutoCommit();
106     conn.setAutoCommit(false);              // bắt đầu giao dịch
107     try (PreparedStatement ps = conn.prepareStatement(sql)) {
108       for (String courseId : courseIds) {
109         ps.setString(1, ssn);
110         ps.setString(2, courseId);
111         ps.executeUpdate();
112       }
113       conn.commit();                        // tất cả thành công
114     } catch (SQLException ex) {
115       conn.rollback();                      // hủy mọi thay đổi
116       throw ex;                             // báo lỗi lên trên
117     } finally {
118       conn.setAutoCommit(oldAutoCommit);    // khôi phục trạng thái
119     }
120   }
121
122   private static Student map(ResultSet rs) throws SQLException {
123     Date d = rs.getDate("birthDate");
124     return new Student(rs.getString("ssn"),
125         rs.getString("firstName"), rs.getString("lastName"),
126         d == null ? null : d.toLocalDate(), rs.getString("deptId"));
127   }
128 }
```

Mã nguồn B.3. DaoDemo.java – chương trình chạy thử DAO

```
1  import java.sql.Connection;
2  import java.sql.SQLException;
3  import java.time.LocalDate;
4  import java.util.List;
5
6  public class DaoDemo {
7    public static void main(String[] args) {
8      String id = "444111199";
9      try (Connection conn = DbConfig.getConnection()) {
10       StudentDao dao = new JdbcStudentDao(conn);  // theo interface
11
12       System.out.println("Sinh vien khoa BIOL:");
13       dao.findByDept("BIOL").forEach(st -> System.out.println("  " + st));
14
15       Student s = new Student(id, "An", "Nguyen",
16           LocalDate.of(2004, 5, 20), "CS");
17       System.out.println("insert -> " + dao.insert(s) + " dong");
18       System.out.println("update -> "
19           + dao.updateDept(id, "MATH") + " dong");
20       System.out.println("Tim lai: "
21           + dao.findBySsn(id).orElse(null));
22
23       try {  // 99999 không có trong Course -> vi phạm khóa ngoại
24         dao.registerCourses(id, List.of("11111", "99999"));
```

```
25        } catch (SQLException ex) {
26          System.out.println("Giao dich bi huy: " + ex.getMessage());
27        }
28        dao.registerCourses(id, List.of("11111", "11112"));
29        System.out.println("Dang ky 2 mon thanh cong");
30        System.out.println("delete -> " + dao.delete(id) + " dong");
31      } catch (SQLException ex) {
32        System.err.println("Loi CSDL: " + ex.getMessage());
33      }
34    }
35  }
```

PHỤ LỤC C. HƯỚNG DẪN BIÊN DỊCH VÀ CHẠY

Các bước dưới đây giúp chạy lại toàn bộ ví dụ trên máy cá nhân và thay các kết quả dự kiến trong Chương 3 bằng kết quả thực tế. Giả sử thư mục dự án gồm src/ chứa các tệp .java, sql/ chứa script và lib/ chứa file JAR của driver.

1. Cài đặt MySQL Server 8 và tải MySQL Connector/J 8 từ trang chủ MySQL; đặt tệp mysql-connector-j-8.x.x.jar vào thư mục lib/.
2. Tạo CSDL và dữ liệu mẫu bằng tài khoản quản trị: mysql -u root -p < sql/javabook.sql.
3. Tạo tài khoản dùng cho ví dụ (nếu muốn dùng tài khoản như giáo trình) bằng hai lệnh SQL trong Mã nguồn C.1.
4. Biên dịch mã nguồn: javac -encoding UTF-8 -d out src/*.java.
5. Chạy chương trình, ví dụ trên Windows: java -cp "out;lib/*" SimpleJdbc; trên Linux hoặc macOS: java -cp "out:lib/*" SimpleJdbc. Thay SimpleJdbc bằng FindGradeDemo, DaoDemo, TestCallableStatement hoặc MetadataDemo để chạy các ví dụ còn lại.
6. Nếu dùng tài khoản khác, truyền thông tin qua thuộc tính hệ thống, ví dụ: java -Ddb.user=root -Ddb.password=matkhau -cp "out;lib/*" DaoDemo.

Mã nguồn C.1. Tạo tài khoản và cấp quyền trên CSDL javabook

```
1  CREATE USER 'scott'@'localhost' IDENTIFIED BY 'tiger';
2  GRANT ALL PRIVILEGES ON javabook.* TO 'scott'@'localhost';
```

Chương trình DaoDemo tự xóa sinh viên thử nghiệm 444111199 ở cuối nên có thể chạy lại nhiều lần. Nếu chương trình bị dừng giữa chừng và để lại dữ liệu, lần chạy sau sẽ báo lỗi trùng khóa chính; khi đó chỉ cần chạy lại script ở bước 2 để khôi phục dữ liệu ban đầu.